13

Prototype Methods and Nearest-Neighbors

13.1 Introduction

In this chapter we discuss some simple and essentially model-free methods for classification and pattern recognition. Because they are highly unstructured, they typically are not useful for understanding the nature of the relationship between the features and class outcome. However, as *black box* prediction engines, they can be very effective, and are often among the best performers in real data problems. The nearest-neighbor technique can also be used in regression; this was touched on in Chapter 2 and works reasonably well for low-dimensional problems. However, with high-dimensional features, the bias–variance tradeoff does not work as favorably for nearest-neighbor regression as it does for classification.

13.2 Prototype Methods

Throughout this chapter, our training data consists of the N pairs (x_1, g_1), $\ldots$, (x_n, g_N) where g_i is a class label taking values in $\{1, 2, \ldots, K\}$. Prototype methods represent the training data by a set of points in feature space. These prototypes are typically not examples from the training sample, except in the case of 1-nearest-neighbor classification discussed later.

Each prototype has an associated class label, and classification of a query point x is made to the class of the closest prototype. "Closest" is usually defined by Euclidean distance in the feature space, after each feature has

been standardized to have overall mean 0 and variance 1 in the training sample. Euclidean distance is appropriate for quantitative features. We discuss distance measures between qualitative and other kinds of feature values in Chapter 14.

These methods can be very effective if the prototypes are well positioned to capture the distribution of each class. Irregular class boundaries can be represented, with enough prototypes in the right places in feature space. The main challenge is to figure out how many prototypes to use and where to put them. Methods differ according to the number and way in which prototypes are selected.

13.2.1 K-means Clustering

K-means clustering is a method for finding clusters and cluster centers in a set of unlabeled data. One chooses the desired number of cluster centers, say R, and the K-means procedure iteratively moves the centers to minimize the total within cluster variance.[1] Given an initial set of centers, the K-means algorithm alternates the two steps:

- for each center we identify the subset of training points (its cluster) that is closer to it than any other center;
- the means of each feature for the data points in each cluster are computed, and this mean vector becomes the new center for that cluster.

These two steps are iterated until convergence. Typically the initial centers are R randomly chosen observations from the training data. Details of the K-means procedure, as well as generalizations allowing for different variable types and more general distance measures, are given in Chapter 14.

To use K-means clustering for classification of labeled data, the steps are:

- apply K-means clustering to the training data in each class separately, using R prototypes per class;
- assign a class label to each of the $K \times R$ prototypes;
- classify a new feature x to the class of the closest prototype.

Figure 13.1 (upper panel) shows a simulated example with three classes and two features. We used $R = 5$ prototypes per class, and show the classification regions and the decision boundary. Notice that a number of the

[1]The "K" in K-means refers to the number of cluster centers. Since we have already reserved K to denote the number of classes, we denote the number of clusters by R.

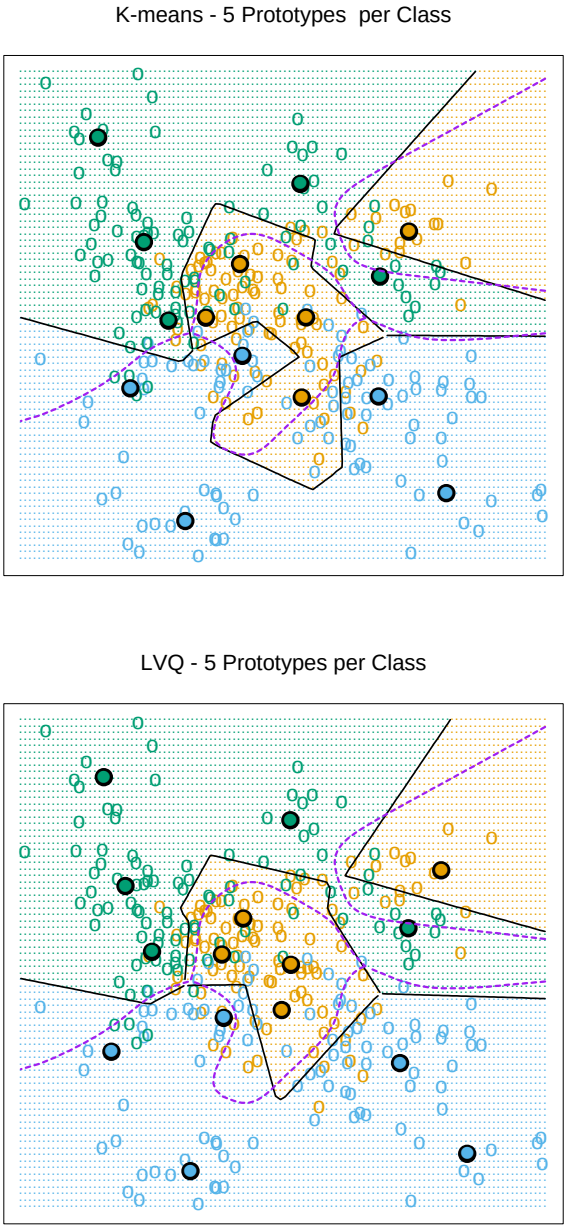

FIGURE 13.1. *Simulated example with three classes and five prototypes per class. The data in each class are generated from a mixture of Gaussians. In the upper panel, the prototypes were found by applying the K-means clustering algorithm separately in each class. In the lower panel, the LVQ algorithm (starting from the K-means solution) moves the prototypes away from the decision boundary. The broken purple curve in the background is the Bayes decision boundary.*

Algorithm 13.1 *Learning Vector Quantization—LVQ.*

1. Choose R initial prototypes for each class: $m_1(k), m_2(k), \ldots, m_R(k)$, $k = 1, 2, \ldots, K$, for example, by sampling R training points at random from each class.

2. Sample a training point x_i randomly (with replacement), and let (j, k) index the closest prototype $m_j(k)$ to x_i.

   (a) If $g_i = k$ (i.e., they are in the same class), move the prototype towards the training point:

   $$m_j(k) \leftarrow m_j(k) + \epsilon(x_i - m_j(k)),$$

   where ϵ is the *learning rate*.

   (b) If $g_i \neq k$ (i.e., they are in different classes), move the prototype away from the training point:

   $$m_j(k) \leftarrow m_j(k) - \epsilon(x_i - m_j(k)).$$

3. Repeat step 2, decreasing the learning rate ϵ with each iteration towards zero.

prototypes are near the class boundaries, leading to potential misclassification errors for points near these boundaries. This results from an obvious shortcoming with this method: for each class, the other classes do not have a say in the positioning of the prototypes for that class. A better approach, discussed next, uses all of the data to position all prototypes.

13.2.2 Learning Vector Quantization

In this technique due to Kohonen (1989), prototypes are placed strategically with respect to the decision boundaries in an ad-hoc way. LVQ is an *online* algorithm—observations are processed one at a time.

The idea is that the training points attract prototypes of the correct class, and repel other prototypes. When the iterations settle down, prototypes should be close to the training points in their class. The learning rate ϵ is decreased to zero with each iteration, following the guidelines for stochastic approximation learning rates (Section 11.4.)

Figure 13.1 (lower panel) shows the result of LVQ, using the K-means solution as starting values. The prototypes have tended to move away from the decision boundaries, and away from prototypes of competing classes.

The procedure just described is actually called LVQ1. Modifications (LVQ2, LVQ3, etc.) have been proposed, that can sometimes improve performance. A drawback of learning vector quantization methods is the fact

that they are defined by algorithms, rather than optimization of some fixed criteria; this makes it difficult to understand their properties.

13.2.3 Gaussian Mixtures

The Gaussian mixture model can also be thought of as a prototype method, similar in spirit to K-means and LVQ. We discuss Gaussian mixtures in some detail in Sections 6.8, 8.5 and 12.7. Each cluster is described in terms of a Gaussian density, which has a centroid (as in K-means), and a covariance matrix. The comparison becomes crisper if we restrict the component Gaussians to have a scalar covariance matrix (Exercise 13.1). The two steps of the alternating EM algorithm are very similar to the two steps in K-means:

- In the E-step, each observation is assigned a *responsibility* or weight for each cluster, based on the likelihood of each of the corresponding Gaussians. Observations close to the center of a cluster will most likely get weight 1 for that cluster, and weight 0 for every other cluster. Observations half-way between two clusters divide their weight accordingly.

- In the M-step, each observation contributes to the weighted means (and covariances) for *every* cluster.

As a consequence, the Gaussian mixture model is often referred to as a *soft* clustering method, while K-means is *hard*.

Similarly, when Gaussian mixture models are used to represent the feature density in each class, it produces smooth posterior probabilities $\hat{p}(x) = \{\hat{p}_1(x), \ldots, \hat{p}_K(x)\}$ for classifying x (see (12.60) on page 449.) Often this is interpreted as a soft classification, while in fact the classification rule is $\hat{G}(x) = \arg\max_k \hat{p}_k(x)$. Figure 13.2 compares the results of K-means and Gaussian mixtures on the simulated mixture problem of Chapter 2. We see that although the decision boundaries are roughly similar, those for the mixture model are smoother (although the prototypes are in approximately the same positions.) We also see that while both procedures devote a blue prototype (incorrectly) to a region in the northwest, the Gaussian mixture classifier can ultimately ignore this region, while K-means cannot. LVQ gave very similar results to K-means on this example, and is not shown.

13.3 k-Nearest-Neighbor Classifiers

These classifiers are *memory-based*, and require no model to be fit. Given a query point x_0, we find the k training points $x_{(r)}, r = 1, \ldots, k$ closest in distance to x_0, and then classify using majority vote among the k neighbors.

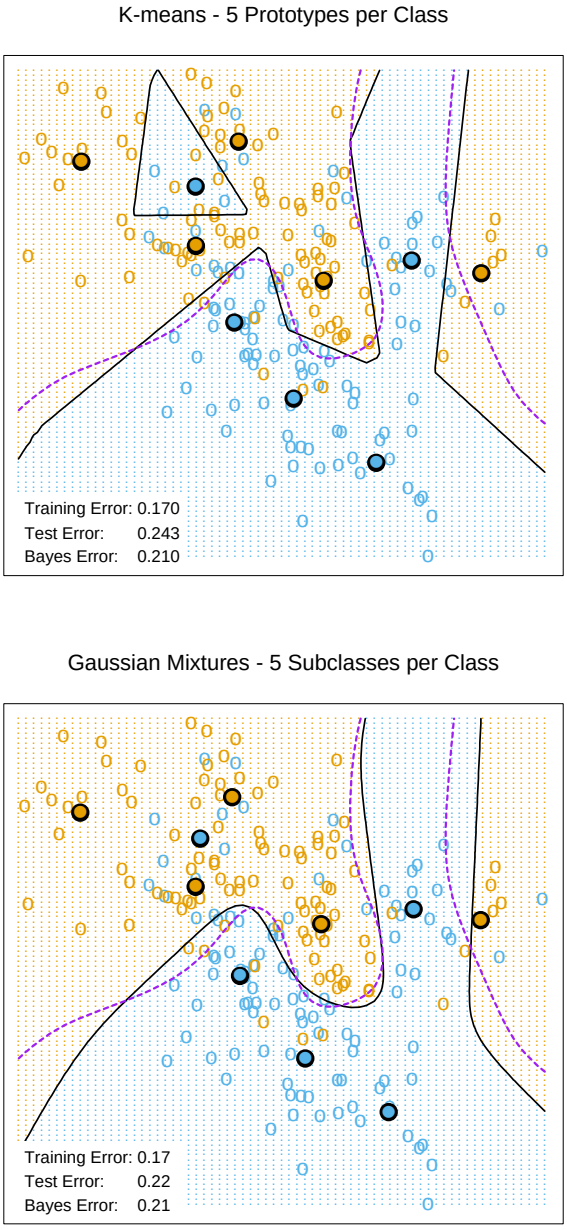

FIGURE 13.2. *The upper panel shows the K-means classifier applied to the mixture data example. The decision boundary is piecewise linear. The lower panel shows a Gaussian mixture model with a common covariance for all component Gaussians. The EM algorithm for the mixture model was started at the K-means solution. The broken purple curve in the background is the Bayes decision boundary.*

Ties are broken at random. For simplicity we will assume that the features are real-valued, and we use Euclidean distance in feature space:

$$d_{(i)} = ||x_{(i)} - x_0||. \tag{13.1}$$

Typically we first standardize each of the features to have mean zero and variance 1, since it is possible that they are measured in different units. In Chapter 14 we discuss distance measures appropriate for qualitative and ordinal features, and how to combine them for mixed data. Adaptively chosen distance metrics are discussed later in this chapter.

Despite its simplicity, k-nearest-neighbors has been successful in a large number of classification problems, including handwritten digits, satellite image scenes and EKG patterns. It is often successful where each class has many possible prototypes, and the decision boundary is very irregular. Figure 13.3 (upper panel) shows the decision boundary of a 15-nearest-neighbor classifier applied to the three-class simulated data. The decision boundary is fairly smooth compared to the lower panel, where a 1-nearest-neighbor classifier was used. There is a close relationship between nearest-neighbor and prototype methods: in 1-nearest-neighbor classification, each training point is a prototype.

Figure 13.4 shows the training, test and tenfold cross-validation errors as a function of the neighborhood size, for the two-class mixture problem. Since the tenfold CV errors are averages of ten numbers, we can estimate a standard error.

Because it uses only the training point closest to the query point, the bias of the 1-nearest-neighbor estimate is often low, but the variance is high. A famous result of Cover and Hart (1967) shows that asymptotically the error rate of the 1-nearest-neighbor classifier is never more than twice the Bayes rate. The rough idea of the proof is as follows (using squared-error loss). We assume that the query point coincides with one of the training points, so that the bias is zero. This is true asymptotically if the dimension of the feature space is fixed and the training data fills up the space in a dense fashion. Then the error of the Bayes rule is just the variance of a Bernoulli random variate (the target at the query point), while the error of 1-nearest-neighbor rule is *twice* the variance of a Bernoulli random variate, one contribution each for the training and query targets.

We now give more detail for misclassification loss. At x let k^* be the dominant class, and $p_k(x)$ the true conditional probability for class k. Then

$$\begin{aligned} \text{Bayes error} &= 1 - p_{k^*}(x), && (13.2)\\ \text{1-nearest-neighbor error} &= \sum_{k=1}^{K} p_k(x)(1 - p_k(x)), && (13.3)\\ &\geq 1 - p_{k^*}(x). && (13.4) \end{aligned}$$

The asymptotic 1-nearest-neighbor error rate is that of a random rule; we pick both the classification and the test point at random with probabili-

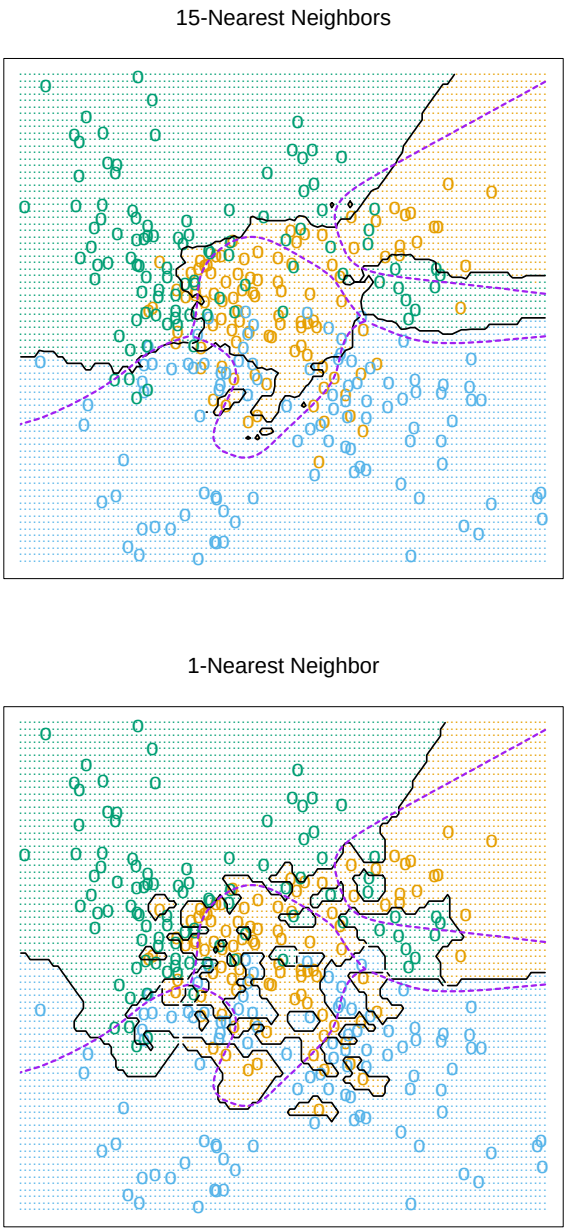

FIGURE 13.3. *k-nearest-neighbor classifiers applied to the simulation data of Figure 13.1. The broken purple curve in the background is the Bayes decision boundary.*

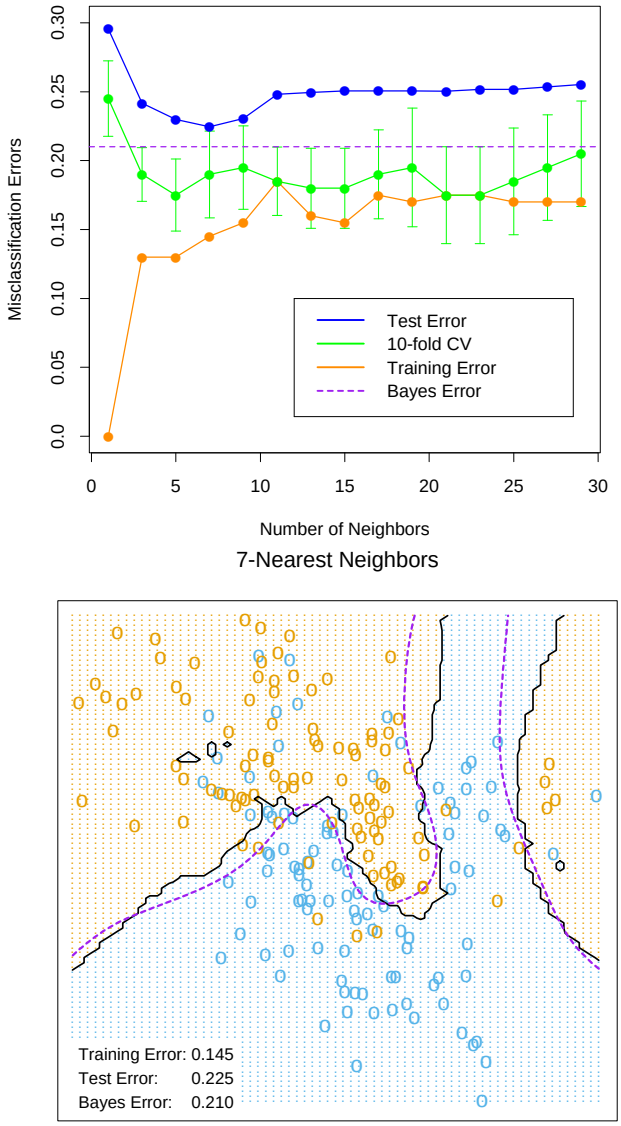

FIGURE 13.4. *k-nearest-neighbors on the two-class mixture data. The upper panel shows the misclassification errors as a function of neighborhood size. Standard error bars are included for 10-fold cross validation. The lower panel shows the decision boundary for 7-nearest-neighbors, which appears to be optimal for minimizing test error. The broken purple curve in the background is the Bayes decision boundary.*

ties $p_k(x)$, $k = 1, \ldots, K$. For $K = 2$ the 1-nearest-neighbor error rate is $2p_{k^*}(x)(1 - p_{k^*}(x)) \leq 2(1 - p_{k^*}(x))$ (twice the Bayes error rate). More generally, one can show (Exercise 13.3)

$$\sum_{k=1}^{K} p_k(x)(1 - p_k(x)) \leq 2(1 - p_{k^*}(x)) - \frac{K}{K-1}(1 - p_{k^*}(x))^2. \tag{13.5}$$

Many additional results of this kind have been derived; Ripley (1996) summarizes a number of them.

This result can provide a rough idea about the best performance that is possible in a given problem. For example, if the 1-nearest-neighbor rule has a 10% error rate, then asymptotically the Bayes error rate is at least 5%. The kicker here is the asymptotic part, which assumes the bias of the nearest-neighbor rule is zero. In real problems the bias can be substantial. The adaptive nearest-neighbor rules, described later in this chapter, are an attempt to alleviate this bias. For simple nearest-neighbors, the bias and variance characteristics can dictate the optimal number of near neighbors for a given problem. This is illustrated in the next example.

13.3.1 Example: A Comparative Study

We tested the nearest-neighbors, K-means and LVQ classifiers on two simulated problems. There are 10 independent features X_j, each uniformly distributed on $[0, 1]$. The two-class 0-1 target variable is defined as follows:

$$\begin{aligned} Y &= I\left(X_1 > \frac{1}{2}\right); \quad \text{problem 1: "easy"}, \\ Y &= I\left(\text{sign}\left\{\prod_{j=1}^{3}\left(X_j - \frac{1}{2}\right)\right\} > 0\right); \quad \text{problem 2: "difficult."} \end{aligned} \tag{13.6}$$

Hence in the first problem the two classes are separated by the hyperplane $X_1 = 1/2$; in the second problem, the two classes form a checkerboard pattern in the hypercube defined by the first three features. The Bayes error rate is zero in both problems. There were 100 training and 1000 test observations.

Figure 13.5 shows the mean and standard error of the misclassification error for nearest-neighbors, K-means and LVQ over ten realizations, as the tuning parameters are varied. We see that K-means and LVQ give nearly identical results. For the best choices of their tuning parameters, K-means and LVQ outperform nearest-neighbors for the first problem, and they perform similarly for the second problem. Notice that the best value of each tuning parameter is clearly situation dependent. For example 25-nearest-neighbors outperforms 1-nearest-neighbor by a factor of 70% in the

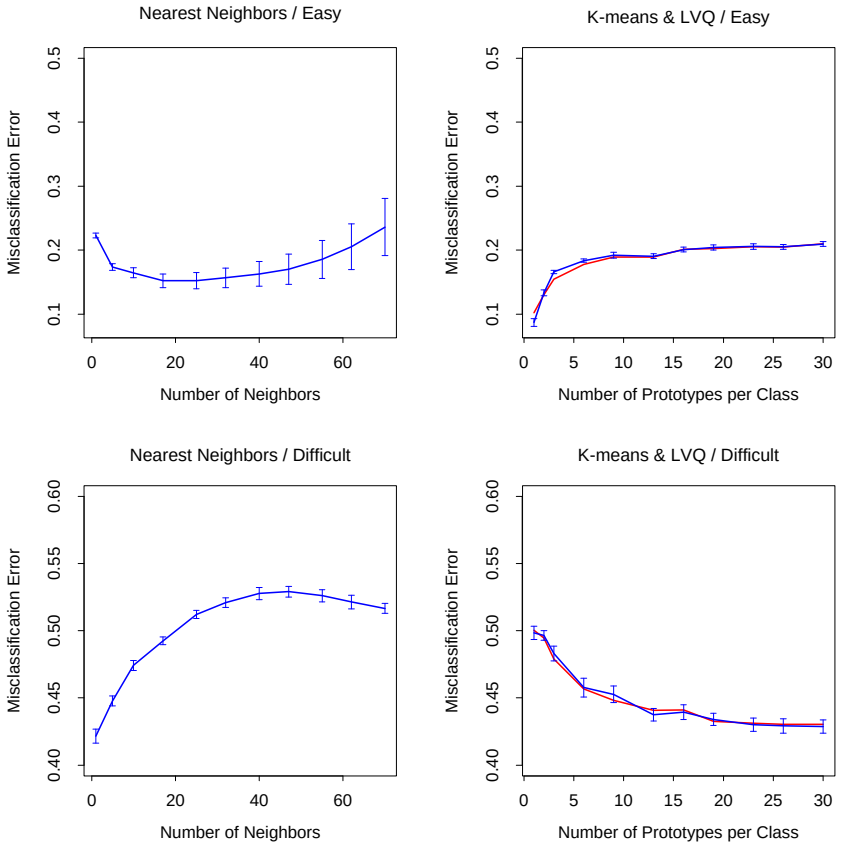

FIGURE 13.5. *Mean ± one standard error of misclassification error for nearest-neighbors, K-means (blue) and LVQ (red) over ten realizations for two simulated problems: "easy" and "difficult," described in the text.*

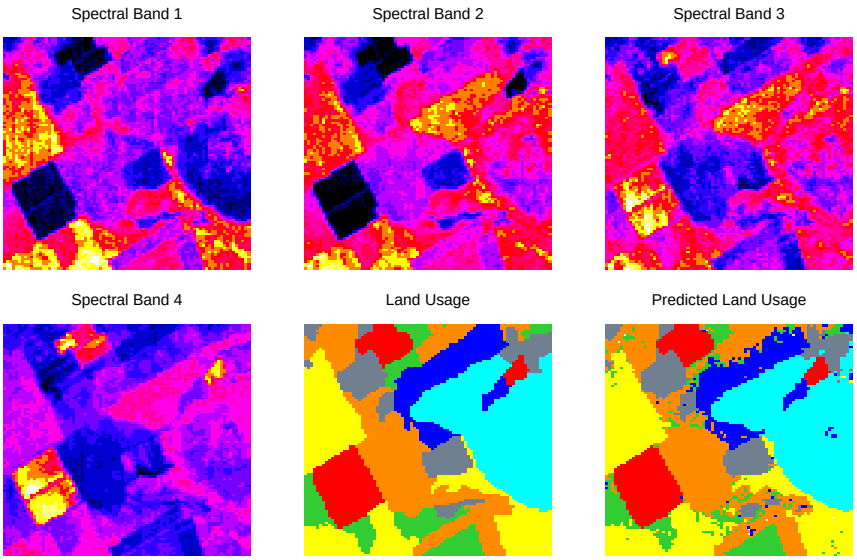

FIGURE 13.6. *The first four panels are LANDSAT images for an agricultural area in four spectral bands, depicted by heatmap shading. The remaining two panels give the actual land usage (color coded) and the predicted land usage using a five-nearest-neighbor rule described in the text.*

first problem, while 1-nearest-neighbor is best in the second problem by a factor of 18%. These results underline the importance of using an objective, data-based method like cross-validation to estimate the best value of a tuning parameter (see Figure 13.4 and Chapter 7).

13.3.2 Example: k-Nearest-Neighbors and Image Scene Classification

The STATLOG project (Michie et al., 1994) used part of a LANDSAT image as a benchmark for classification (82×100 pixels). Figure 13.6 shows four heat-map images, two in the visible spectrum and two in the infrared, for an area of agricultural land in Australia. Each pixel has a class label from the 7-element set $\mathcal{G} = \{$*red soil, cotton, vegetation stubble, mixture, gray soil, damp gray soil, very damp gray soil*$\}$, determined manually by research assistants surveying the area. The lower middle panel shows the actual land usage, shaded by different colors to indicate the classes. The objective is to classify the land usage at a pixel, based on the information in the four spectral bands.

Five-nearest-neighbors produced the predicted map shown in the bottom right panel, and was computed as follows. For each pixel we extracted an 8-neighbor feature map—the pixel itself and its 8 immediate neighbors

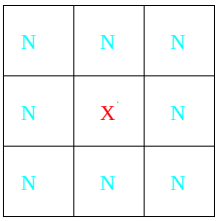

FIGURE 13.7. *A pixel and its* 8*-neighbor feature map.*

(see Figure 13.7). This is done separately in the four spectral bands, giving $(1+8)\times 4 = 36$ input features per pixel. Then five-nearest-neighbors classification was carried out in this 36-dimensional feature space. The resulting test error rate was about 9.5% (see Figure 13.8). Of all the methods used in the STATLOG project, including LVQ, CART, neural networks, linear discriminant analysis and many others, k-nearest-neighbors performed best on this task. Hence it is likely that the decision boundaries in $\mathbb{R}^{36}$ are quite irregular.

13.3.3 Invariant Metrics and Tangent Distance

In some problems, the training features are invariant under certain natural transformations. The nearest-neighbor classifier can exploit such invariances by incorporating them into the metric used to measure the distances between objects. Here we give an example where this idea was used with great success, and the resulting classifier outperformed all others at the time of its development (Simard et al., 1993).

The problem is handwritten digit recognition, as discussed is Chapter 1 and Section 11.7. The inputs are grayscale images with $16 \times 16 = 256$ pixels; some examples are shown in Figure 13.9. At the top of Figure 13.10, a "3" is shown, in its actual orientation (middle) and rotated 7.5° and 15° in either direction. Such rotations can often occur in real handwriting, and it is obvious to our eye that this "3" is still a "3" after small rotations. Hence we want our nearest-neighbor classifier to consider these two "3"s to be close together (similar). However the 256 grayscale pixel values for a rotated "3" will look quite different from those in the original image, and hence the two objects can be far apart in Euclidean distance in $\mathbb{R}^{256}$.

We wish to remove the effect of rotation in measuring distances between two digits of the same class. Consider the set of pixel values consisting of the original "3" and its rotated versions. This is a one-dimensional curve in $\mathbb{R}^{256}$, depicted by the green curve passing through the "3" in Figure 13.10. Figure 13.11 shows a stylized version of $\mathbb{R}^{256}$, with two images indicated by x_i and $x_{i'}$. These might be two different "3"s, for example. Through each image we have drawn the curve of rotated versions of that image, called

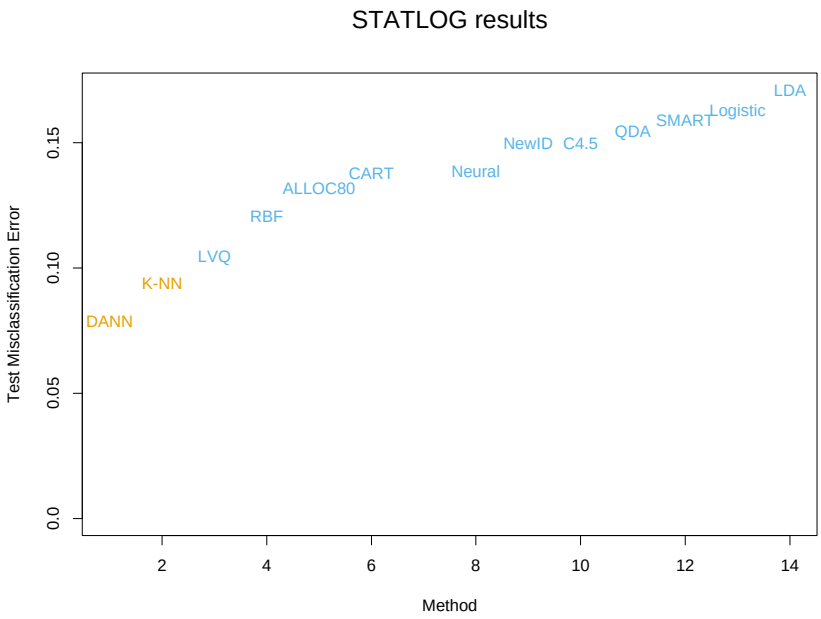

FIGURE 13.8. *Test-error performance for a number of classifiers, as reported by the STATLOG project. The entry DANN is a variant of k-nearest neighbors, using an adaptive metric (Section 13.4.2).*

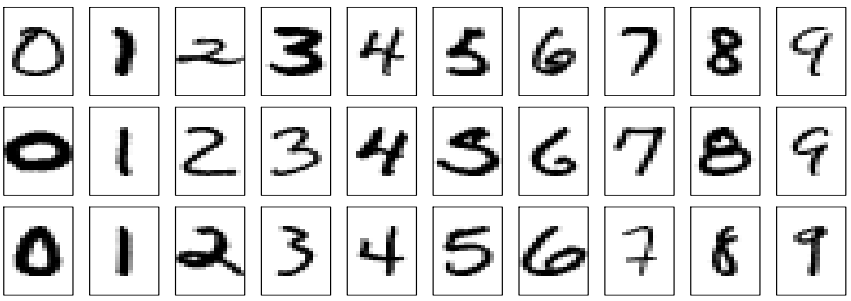

FIGURE 13.9. *Examples of grayscale images of handwritten digits.*

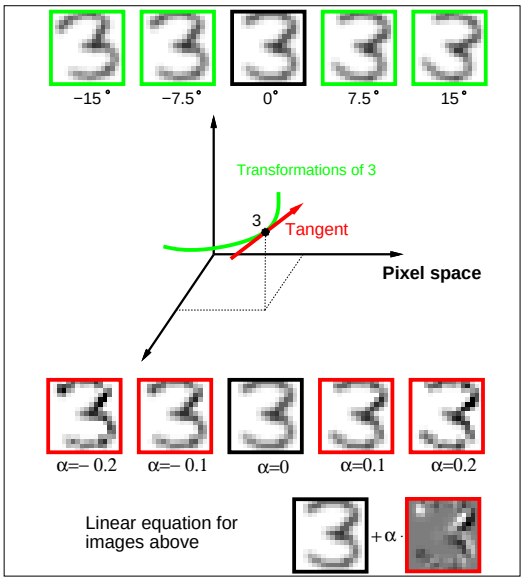

FIGURE 13.10. *The top row shows a "3" in its original orientation (middle) and rotated versions of it. The green curve in the middle of the figure depicts this set of rotated "3" in 256-dimensional space. The red line is the tangent line to the curve at the original image, with some "3"s on this tangent line, and its equation shown at the bottom of the figure.*

invariance manifolds in this context. Now, rather than using the usual Euclidean distance between the two images, we use the shortest distance between the two curves. In other words, the distance between the two images is taken to be the shortest Euclidean distance between any rotated version of first image, and any rotated version of the second image. This distance is called an *invariant metric*.

In principle one could carry out 1-nearest-neighbor classification using this invariant metric. However there are two problems with it. First, it is very difficult to calculate for real images. Second, it allows large transformations that can lead to poor performance. For example a "6" would be considered close to a "9" after a rotation of 180°. We need to restrict attention to small rotations.

The use of *tangent distance* solves both of these problems. As shown in Figure 13.10, we can approximate the invariance manifold of the image "3" by its tangent at the original image. This tangent can be computed by estimating the direction vector from small rotations of the image, or by more sophisticated spatial smoothing methods (Exercise 13.4.) For large rotations, the tangent image no longer looks like a "3," so the problem with large transformations is alleviated.

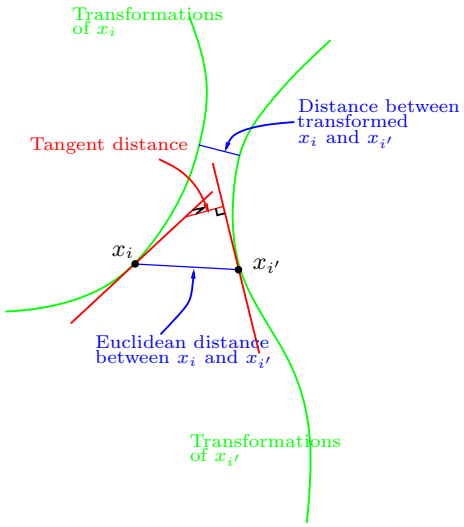

FIGURE 13.11. *Tangent distance computation for two images x_i and $x_{i'}$. Rather than using the Euclidean distance between x_i and $x_{i'}$, or the shortest distance between the two curves, we use the shortest distance between the two tangent lines.*

The idea then is to compute the invariant tangent line for each training image. For a query image to be classified, we compute its invariant tangent line, and find the closest line to it among the lines in the training set. The class (digit) corresponding to this closest line is our predicted class for the query image. In Figure 13.11 the two tangent lines intersect, but this is only because we have been forced to draw a two-dimensional representation of the actual 256-dimensional situation. In $\mathbb{R}^{256}$ the probability of two such lines intersecting is effectively zero.

Now a simpler way to achieve this invariance would be to add into the training set a number of rotated versions of each training image, and then just use a standard nearest-neighbor classifier. This idea is called "hints" in Abu-Mostafa (1995), and works well when the space of invariances is small. So far we have presented a simplified version of the problem. In addition to rotation, there are six other types of transformations under which we would like our classifier to be invariant. There are translation (two directions), scaling (two directions), sheer, and character thickness. Hence the curves and tangent lines in Figures 13.10 and 13.11 are actually 7-dimensional manifolds and hyperplanes. It is infeasible to add transformed versions of each training image to capture all of these possibilities. The tangent manifolds provide an elegant way of capturing the invariances.

Table 13.1 shows the test misclassification error for a problem with 7291 training images and 2007 test digits (the U.S. Postal Services database), for a carefully constructed neural network, and simple 1-nearest-neighbor and

TABLE 13.1. *Test error rates for the handwritten ZIP code problem.*

Method	Error rate
Neural-net	0.049
1-nearest-neighbor/Euclidean distance	0.055
1-nearest-neighbor/tangent distance	0.026

tangent distance 1-nearest-neighbor rules. The tangent distance nearest-neighbor classifier works remarkably well, with test error rates near those for the human eye (this is a notoriously difficult test set). In practice, it turned out that nearest-neighbors are too slow for online classification in this application (see Section 13.5), and neural network classifiers were subsequently developed to mimic it.

13.4 Adaptive Nearest-Neighbor Methods

When nearest-neighbor classification is carried out in a high-dimensional feature space, the nearest neighbors of a point can be very far away, causing bias and degrading the performance of the rule.

To quantify this, consider N data points uniformly distributed in the unit cube $[-\frac{1}{2}, \frac{1}{2}]^p$. Let R be the radius of a 1-nearest-neighborhood centered at the origin. Then

$$\text{median}(R) = v_p^{-1/p}\left(1 - \frac{1}{2}^{1/N}\right)^{1/p}, \tag{13.7}$$

where $v_p r^p$ is the volume of the sphere of radius r in p dimensions. Figure 13.12 shows the median radius for various training sample sizes and dimensions. We see that median radius quickly approaches 0.5, the distance to the edge of the cube.

What can be done about this problem? Consider the two-class situation in Figure 13.13. There are two features, and a nearest-neighborhood at a query point is depicted by the circular region. Implicit in near-neighbor classification is the assumption that the class probabilities are roughly constant in the neighborhood, and hence simple averages give good estimates. However, in this example the class probabilities vary only in the horizontal direction. If we knew this, we would stretch the neighborhood in the vertical direction, as shown by the tall rectangular region. This will reduce the bias of our estimate and leave the variance the same.

In general, this calls for adapting the metric used in nearest-neighbor classification, so that the resulting neighborhoods stretch out in directions for which the class probabilities don't change much. In high-dimensional feature space, the class probabilities might change only a low-dimensional subspace and hence there can be considerable advantage to adapting the metric.

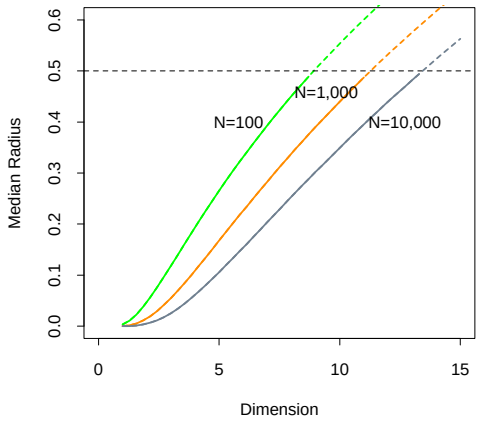

FIGURE 13.12. *Median radius of a 1-nearest-neighborhood, for uniform data with N observations in p dimensions.*

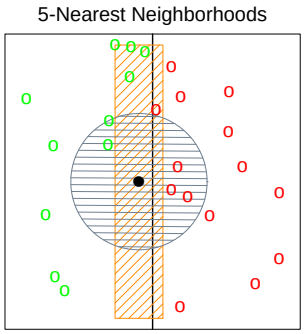

FIGURE 13.13. *The points are uniform in the cube, with the vertical line separating class red and green. The vertical strip denotes the 5-nearest-neighbor region using only the horizontal coordinate to find the nearest-neighbors for the target point (solid dot). The sphere shows the 5-nearest-neighbor region using both coordinates, and we see in this case it has extended into the class-red region (and is dominated by the wrong class in this instance).*

Friedman (1994a) proposed a method in which rectangular neighborhoods are found adaptively by successively carving away edges of a box containing the training data. Here we describe the *discriminant adaptive nearest-neighbor* (DANN) rule of Hastie and Tibshirani (1996a). Earlier, related proposals appear in Short and Fukunaga (1981) and Myles and Hand (1990).

At each query point a neighborhood of say 50 points is formed, and the class distribution among the points is used to decide how to deform the neighborhood—that is, to adapt the metric. The adapted metric is then used in a nearest-neighbor rule at the query point. Thus at each query point a potentially different metric is used.

In Figure 13.13 it is clear that the neighborhood should be stretched in the direction orthogonal to line joining the class centroids. This direction also coincides with the linear discriminant boundary, and is the direction in which the class probabilities change the least. In general this direction of maximum change will not be orthogonal to the line joining the class centroids (see Figure 4.9 on page 116.) Assuming a local discriminant model, the information contained in the local within- and between-class covariance matrices is all that is needed to determine the optimal shape of the neighborhood.

The *discriminant adaptive nearest-neighbor* (DANN) metric at a query point x_0 is defined by

$$D(x, x_0) = (x - x_0)^T \mathbf{\Sigma} (x - x_0), \tag{13.8}$$

where

$$\begin{aligned} \mathbf{\Sigma} &= \mathbf{W}^{-1/2}[\mathbf{W}^{-1/2}\mathbf{B}\mathbf{W}^{-1/2} + \epsilon \mathbf{I}]\mathbf{W}^{-1/2} \\ &= \mathbf{W}^{-1/2}[\mathbf{B}^* + \epsilon \mathbf{I}]\mathbf{W}^{-1/2}. \end{aligned} \tag{13.9}$$

Here $\mathbf{W}$ is the pooled within-class covariance matrix $\sum_{k=1}^{K} \pi_k \mathbf{W}_k$ and $\mathbf{B}$ is the between class covariance matrix $\sum_{k=1}^{K} \pi_k (\bar{x}_k - \bar{x})(\bar{x}_k - \bar{x})^T$, with $\mathbf{W}$ and $\mathbf{B}$ computed using only the 50 nearest neighbors around x_0. After computation of the metric, it is used in a nearest-neighbor rule at x_0.

This complicated formula is actually quite simple in its operation. It first spheres the data with respect to $\mathbf{W}$, and then stretches the neighborhood in the zero-eigenvalue directions of $\mathbf{B}^*$ (the between-matrix for the sphered data). This makes sense, since locally the observed class means do not differ in these directions. The ϵ parameter rounds the neighborhood, from an infinite strip to an ellipsoid, to avoid using points far away from the query point. The value of $\epsilon = 1$ seems to work well in general. Figure 13.14 shows the resulting neighborhoods for a problem where the classes form two concentric circles. Notice how the neighborhoods stretch out orthogonally to the decision boundaries when both classes are present in the neighborhood. In the pure regions with only one class, the neighborhoods remain circular;

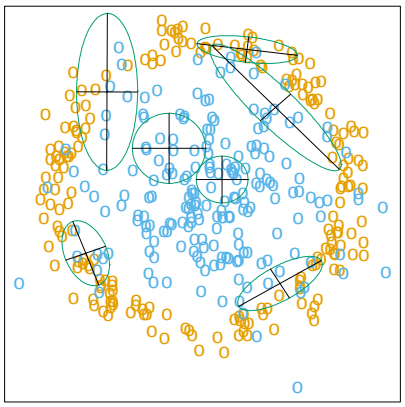

FIGURE 13.14. *Neighborhoods found by the DANN procedure, at various query points (centers of the crosses). There are two classes in the data, with one class surrounding the other.* 50 *nearest-neighbors were used to estimate the local metrics. Shown are the resulting metrics used to form* 15*-nearest-neighborhoods.*

in these cases the between matrix $\mathbf{B} = 0$, and the $\mathbf{\Sigma}$ in (13.8) is the identity matrix.

13.4.1 Example

Here we generate two-class data in ten dimensions, analogous to the two-dimensional example of Figure 13.14. All ten predictors in class 1 are independent standard normal, conditioned on the squared radius being greater than 22.4 and less than 40, while the predictors in class 2 are independent standard normal without the restriction. There are 250 observations in each class. Hence the first class almost completely surrounds the second class in the full ten-dimensional space.

In this example there are no pure noise variables, the kind that a nearest-neighbor subset selection rule might be able to weed out. At any given point in the feature space, the class discrimination occurs along only one direction. However, this direction changes as we move across the feature space and all variables are important somewhere in the space.

Figure 13.15 shows boxplots of the test error rates over ten realizations, for standard 5-nearest-neighbors, LVQ, and discriminant adaptive 5-nearest-neighbors. We used 50 prototypes per class for LVQ, to make it comparable to 5 nearest-neighbors (since $250/5 = 50$). The adaptive metric significantly reduces the error rate, compared to LVQ or standard nearest-neighbors.

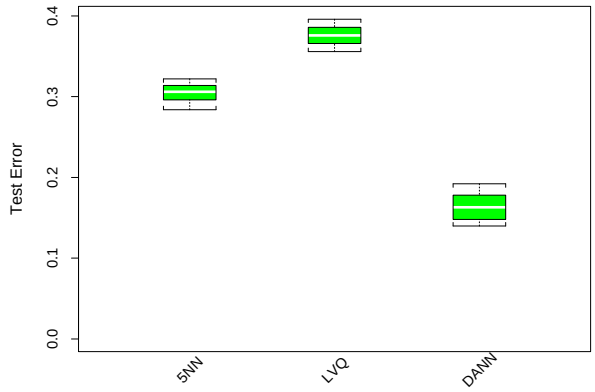

FIGURE 13.15. *Ten-dimensional simulated example: boxplots of the test error rates over ten realizations, for standard* 5*-nearest-neighbors, LVQ with* 50 *centers, and discriminant-adaptive* 5*-nearest-neighbors*

13.4.2 Global Dimension Reduction for Nearest-Neighbors

The discriminant-adaptive nearest-neighbor method carries out local dimension reduction—that is, dimension reduction separately at each query point. In many problems we can also benefit from global dimension reduction, that is, apply a nearest-neighbor rule in some optimally chosen subspace of the original feature space. For example, suppose that the two classes form two nested spheres in four dimensions of feature space, and there are an additional six noise features whose distribution is independent of class. Then we would like to discover the important four-dimensional subspace, and carry out nearest-neighbor classification in that reduced subspace. Hastie and Tibshirani (1996a) discuss a variation of the discriminant-adaptive nearest-neighbor method for this purpose. At each training point x_i, the between-centroids sum of squares matrix $\mathbf{B}_i$ is computed, and then these matrices are averaged over all training points:

$$\bar{\mathbf{B}} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{B}_i. \tag{13.10}$$

Let $e_1, e_2, \ldots, e_p$ be the eigenvectors of the matrix $\bar{\mathbf{B}}$, ordered from largest to smallest eigenvalue θ_k. Then these eigenvectors span the optimal subspaces for global subspace reduction. The derivation is based on the fact that the best rank-L approximation to $\bar{\mathbf{B}}$, $\bar{\mathbf{B}}_{[L]} = \sum_{\ell=1}^{L}\theta_\ell e_\ell e_\ell^T$, solves the least squares problem

$$\min_{\text{rank}(\mathbf{M})=L}\sum_{i=1}^{N}\text{trace}[(\mathbf{B}_i-\mathbf{M})^2]. \tag{13.11}$$

Since each $\mathbf{B}_i$ contains information on (a) the local discriminant subspace, and (b) the strength of discrimination in that subspace, (13.11) can be seen

as a way of finding the best approximating subspace of dimension L to a series of N subspaces by weighted least squares (Exercise 13.5.)

In the four-dimensional sphere example mentioned above and examined in Hastie and Tibshirani (1996a), four of the eigenvalues θ_ℓ turn out to be large (having eigenvectors nearly spanning the interesting subspace), and the remaining six are near zero. Operationally, we project the data into the leading four-dimensional subspace, and then carry out nearest neighbor classification. In the satellite image classification example in Section 13.3.2, the technique labeled `DANN` in Figure 13.8 used 5-nearest-neighbors in a globally reduced subspace. There are also connections of this technique with the *sliced inverse regression* proposal of Duan and Li (1991). These authors use similar ideas in the regression setting, but do global rather than local computations. They assume and exploit spherical symmetry of the feature distribution to estimate interesting subspaces.

13.5 Computational Considerations

One drawback of nearest-neighbor rules in general is the computational load, both in finding the neighbors and storing the entire training set. With N observations and p predictors, nearest-neighbor classification requires Np operations to find the neighbors per query point. There are fast algorithms for finding nearest-neighbors (Friedman et al., 1975; Friedman et al., 1977) which can reduce this load somewhat. Hastie and Simard (1998) reduce the computations for tangent distance by developing analogs of K-means clustering in the context of this invariant metric.

Reducing the storage requirements is more difficult, and various *editing* and *condensing* procedures have been proposed. The idea is to isolate a subset of the training set that suffices for nearest-neighbor predictions, and throw away the remaining training data. Intuitively, it seems important to keep the training points that are near the decision boundaries and on the correct side of those boundaries, while some points far from the boundaries could be discarded.

The *multi-edit* algorithm of Devijver and Kittler (1982) divides the data cyclically into training and test sets, computing a nearest neighbor rule on the training set and deleting test points that are misclassified. The idea is to keep homogeneous clusters of training observations.

The *condensing* procedure of Hart (1968) goes further, trying to keep only important exterior points of these clusters. Starting with a single randomly chosen observation as the training set, each additional data item is processed one at a time, adding it to the training set only if it is misclassified by a nearest-neighbor rule computed on the current training set.

These procedures are surveyed in Dasarathy (1991) and Ripley (1996). They can also be applied to other learning procedures besides nearest-

neighbors. While such methods are sometimes useful, we have not had much practical experience with them, nor have we found any systematic comparison of their performance in the literature.

Bibliographic Notes

The nearest-neighbor method goes back at least to Fix and Hodges (1951). The extensive literature on the topic is reviewed by Dasarathy (1991); Chapter 6 of Ripley (1996) contains a good summary. K-means clustering is due to Lloyd (1957) and MacQueen (1967). Kohonen (1989) introduced learning vector quantization. The tangent distance method is due to Simard et al. (1993). Hastie and Tibshirani (1996a) proposed the discriminant adaptive nearest-neighbor technique.

Exercises

Ex. 13.1 Consider a Gaussian mixture model where the covariance matrices are assumed to be scalar: $\mathbf{\Sigma}_r = \sigma\mathbf{I}\ \forall r = 1, \ldots, R$, and σ is a fixed parameter. Discuss the analogy between the K-means clustering algorithm and the EM algorithm for fitting this mixture model in detail. Show that in the limit $\sigma \to 0$ the two methods coincide.

Ex. 13.2 Derive formula (13.7) for the median radius of the 1-nearest-neighborhood.

Ex. 13.3 Let E^* be the error rate of the Bayes rule in a K-class problem, where the true class probabilities are given by $p_k(x),\ k = 1, \ldots, K$. Assuming the test point and training point have identical features x, prove (13.5)

$$\sum_{k=1}^{K} p_k(x)(1 - p_k(x)) \leq 2(1 - p_{k^*}(x)) - \frac{K}{K-1}(1 - p_{k^*}(x))^2.$$

where $k^* = \arg\max_k p_k(x)$. Hence argue that the error rate of the 1-nearest-neighbor rule converges in L_1, as the size of the training set increases, to a value E_1, bounded above by

$$E^*\Big(2 - E^*\frac{K}{K-1}\Big). \tag{13.12}$$

[This statement of the theorem of Cover and Hart (1967) is taken from Chapter 6 of Ripley (1996), where a short proof is also given].

Ex. 13.4 Consider an image to be a function $F(x) : \mathbb{R}^2 \mapsto \mathbb{R}^1$ over the two-dimensional spatial domain (paper coordinates). Then $F(c+x_0+\mathbf{A}(x-x_0))$ represents an affine transformation of the image F, where $\mathbf{A}$ is a 2×2 matrix.

1. Decompose $\mathbf{A}$ (via Q-R) in such a way that parameters identifying the four affine transformations (two scale, shear and rotation) are clearly identified.

2. Using the chain rule, show that the derivative of $F(c+x_0+\mathbf{A}(x-x_0))$ w.r.t. each of these parameters can be represented in terms of the two spatial derivatives of F.

3. Using a two-dimensional kernel smoother (Chapter 6), describe how to implement this procedure when the images are quantized to 16×16 pixels.

Ex. 13.5 Let $\mathbf{B}_i, i = 1, 2, \ldots, N$ be square $p \times p$ positive semi-definite matrices and let $\bar{\mathbf{B}} = (1/N) \sum \mathbf{B}_i$. Write the eigen-decomposition of $\bar{\mathbf{B}}$ as $\sum_{\ell=1}^{p} \theta_\ell e_\ell e_\ell^T$ with $\theta_\ell \geq \theta_{\ell-1} \geq \cdots \geq \theta_1$. Show that the best rank-L approximation for the $\mathbf{B}_i$,

$$\min_{\text{rank}(\mathbf{M})=L} \sum_{i=1}^{N} \text{trace}[(\mathbf{B}_i - \mathbf{M})^2],$$

is given by $\bar{\mathbf{B}}_{[L]} = \sum_{\ell=1}^{L} \theta_\ell e_\ell e_\ell^T$. (*Hint:* Write $\sum_{i=1}^{N} \text{trace}[(\mathbf{B}_i - \mathbf{M})^2]$ as

$$\sum_{i=1}^{N} \text{trace}[(\mathbf{B}_i - \bar{\mathbf{B}})^2] + \sum_{i=1}^{N} \text{trace}[(\mathbf{M} - \bar{\mathbf{B}})^2]).$$

Ex. 13.6 Here we consider the problem of *shape averaging*. In particular, $\mathbf{L}_i,\ i = 1, \ldots, M$ are each $N \times 2$ matrices of points in $\mathbb{R}^2$, each sampled from corresponding positions of handwritten (cursive) letters. We seek an *affine invariant average* $\mathbf{V}$, also $N \times 2$, $\mathbf{V}^T\mathbf{V} = I$, of the M letters $\mathbf{L}_i$ with the following property: $\mathbf{V}$ minimizes

$$\sum_{j=1}^{M} \min_{\mathbf{A}_j} \|\mathbf{L}_j - \mathbf{V}\mathbf{A}_j\|^2 .$$

Characterize the solution.

This solution can suffer if some of the letters are *big* and dominate the average. An alternative approach is to minimize instead:

$$\sum_{j=1}^{M} \min_{\mathbf{A}_j} \left\|\mathbf{L}_j\mathbf{A}_j^* - \mathbf{V}\right\|^2 .$$

Derive the solution to this problem. How do the criteria differ? Use the SVD of the $\mathbf{L}_j$ to simplify the comparison of the two approaches.

Ex. 13.7 Consider the application of nearest-neighbors to the "easy" and "hard" problems in the left panel of Figure 13.5.

1. Replicate the results in the left panel of Figure 13.5.
2. Estimate the misclassification errors using fivefold cross-validation, and compare the error rate curves to those in 1.
3. Consider an "AIC-like" penalization of the training set misclassification error. Specifically, add $2t/N$ to the training set misclassification error, where t is the approximate number of parameters N/r, r being the number of nearest-neighbors. Compare plots of the resulting penalized misclassification error to those in 1 and 2. Which method gives a better estimate of the optimal number of nearest-neighbors: cross-validation or AIC?

Ex. 13.8 Generate data in two classes, with two features. These features are all independent Gaussian variates with standard deviation 1. Their mean vectors are $(-1, -1)$ in class 1 and $(1, 1)$ in class 2. To each feature vector apply a random rotation of angle θ, θ chosen uniformly from 0 to 2π. Generate 50 observations from each class to form the training set, and 500 in each class as the test set. Apply four different classifiers:

1. Nearest-neighbors.
2. Nearest-neighbors with hints: ten randomly rotated versions of each data point are added to the training set before applying nearest-neighbors.
3. Invariant metric nearest-neighbors, using Euclidean distance invariant to rotations about the origin.
4. Tangent distance nearest-neighbors.

In each case choose the number of neighbors by tenfold cross-validation. Compare the results.

14
Unsupervised Learning

14.1 Introduction

The previous chapters have been concerned with predicting the values of one or more outputs or response variables $Y = (Y_1, \ldots, Y_m)$ for a given set of input or predictor variables $X^T = (X_1, \ldots, X_p)$. Denote by $x_i^T = (x_{i1}, \ldots, x_{ip})$ the inputs for the ith training case, and let y_i be a response measurement. The predictions are based on the training sample $(x_1, y_1), \ldots, (x_N, y_N)$ of previously solved cases, where the joint values of all of the variables are known. This is called *supervised learning* or "learning with a teacher." Under this metaphor the "student" presents an answer $\hat{y}_i$ for each x_i in the training sample, and the supervisor or "teacher" provides either the correct answer and/or an error associated with the student's answer. This is usually characterized by some loss function $L(y, \hat{y})$, for example, $L(y, \hat{y}) = (y - \hat{y})^2$.

If one supposes that (X, Y) are random variables represented by some joint probability density $\Pr(X, Y)$, then supervised learning can be formally characterized as a density estimation problem where one is concerned with determining properties of the conditional density $\Pr(Y|X)$. Usually the properties of interest are the "location" parameters μ that minimize the expected error at each x,

$$\mu(x) = \underset{\theta}{\operatorname{argmin}}\, E_{Y|X} L(Y, \theta). \tag{14.1}$$

Conditioning one has

$$\Pr(X, Y) = \Pr(Y|X) \cdot \Pr(X),$$

where $\Pr(X)$ is the joint marginal density of the X values alone. In supervised learning $\Pr(X)$ is typically of no direct concern. One is interested mainly in the properties of the conditional density $\Pr(Y|X)$. Since Y is often of low dimension (usually one), and only its location $\mu(x)$ is of interest, the problem is greatly simplified. As discussed in the previous chapters, there are many approaches for successfully addressing supervised learning in a variety of contexts.

In this chapter we address *unsupervised learning* or "learning without a teacher." In this case one has a set of N observations $(x_1, x_2, \ldots, x_N)$ of a random p-vector X having joint density $\Pr(X)$. The goal is to directly infer the properties of this probability density without the help of a supervisor or teacher providing correct answers or degree-of-error for each observation. The dimension of X is sometimes much higher than in supervised learning, and the properties of interest are often more complicated than simple location estimates. These factors are somewhat mitigated by the fact that X represents all of the variables under consideration; one is not required to infer how the properties of $\Pr(X)$ change, conditioned on the changing values of another set of variables.

In low-dimensional problems (say $p \leq 3$), there are a variety of effective nonparametric methods for directly estimating the density $\Pr(X)$ itself at all X-values, and representing it graphically (Silverman, 1986, e.g.). Owing to the curse of dimensionality, these methods fail in high dimensions. One must settle for estimating rather crude global models, such as Gaussian mixtures or various simple descriptive statistics that characterize $\Pr(X)$.

Generally, these descriptive statistics attempt to characterize X-values, or collections of such values, where $\Pr(X)$ is relatively large. Principal components, multidimensional scaling, self-organizing maps, and principal curves, for example, attempt to identify low-dimensional manifolds within the X-space that represent high data density. This provides information about the associations among the variables and whether or not they can be considered as functions of a smaller set of "latent" variables. Cluster analysis attempts to find multiple convex regions of the X-space that contain modes of $\Pr(X)$. This can tell whether or not $\Pr(X)$ can be represented by a mixture of simpler densities representing distinct types or classes of observations. Mixture modeling has a similar goal. Association rules attempt to construct simple descriptions (conjunctive rules) that describe regions of high density in the special case of very high dimensional binary-valued data.

With supervised learning there is a clear measure of success, or lack thereof, that can be used to judge adequacy in particular situations and to compare the effectiveness of different methods over various situations.

Lack of success is directly measured by expected loss over the joint distribution $\Pr(X, Y)$. This can be estimated in a variety of ways including cross-validation. In the context of unsupervised learning, there is no such direct measure of success. It is difficult to ascertain the validity of inferences drawn from the output of most unsupervised learning algorithms. One must resort to heuristic arguments not only for motivating the algorithms, as is often the case in supervised learning as well, but also for judgments as to the quality of the results. This uncomfortable situation has led to heavy proliferation of proposed methods, since effectiveness is a matter of opinion and cannot be verified directly.

In this chapter we present those unsupervised learning techniques that are among the most commonly used in practice, and additionally, a few others that are favored by the authors.

14.2 Association Rules

Association rule analysis has emerged as a popular tool for mining commercial data bases. The goal is to find joint values of the variables $X = (X_1, X_2, \ldots, X_p)$ that appear most frequently in the data base. It is most often applied to binary-valued data $X_j \in \{0, 1\}$, where it is referred to as "market basket" analysis. In this context the observations are sales transactions, such as those occurring at the checkout counter of a store. The variables represent all of the items sold in the store. For observation i, each variable X_j is assigned one of two values; $x_{ij} = 1$ if the jth item is purchased as part of the transaction, whereas $x_{ij} = 0$ if it was not purchased. Those variables that frequently have joint values of one represent items that are frequently purchased together. This information can be quite useful for stocking shelves, cross-marketing in sales promotions, catalog design, and consumer segmentation based on buying patterns.

More generally, the basic goal of association rule analysis is to find a collection of prototype X-values $v_1, \ldots, v_L$ for the feature vector X, such that the probability density $\Pr(v_l)$ evaluated at each of those values is relatively large. In this general framework, the problem can be viewed as "mode finding" or "bump hunting." As formulated, this problem is impossibly difficult. A natural estimator for each $\Pr(v_l)$ is the fraction of observations for which $X = v_l$. For problems that involve more than a small number of variables, each of which can assume more than a small number of values, the number of observations for which $X = v_l$ will nearly always be too small for reliable estimation. In order to have a tractable problem, both the goals of the analysis and the generality of the data to which it is applied must be greatly simplified.

The first simplification modifies the goal. Instead of seeking *values* x where $\Pr(x)$ is large, one seeks *regions* of the X-space with high probability

content relative to their size or support. Let $\mathcal{S}_j$ represent the set of all possible values of the jth variable (its *support*), and let $s_j \subseteq \mathcal{S}_j$ be a subset of these values. The modified goal can be stated as attempting to find subsets of variable values $s_1, \ldots, s_p$ such that the probability of each of the variables simultaneously assuming a value within its respective subset,

$$\Pr\left[\bigcap_{j=1}^{p}(X_j \in s_j)\right], \tag{14.2}$$

is relatively large. The intersection of subsets $\cap_{j=1}^{p}(X_j \in s_j)$ is called a *conjunctive rule*. For quantitative variables the subsets s_j are contiguous intervals; for categorical variables the subsets are delineated explicitly. Note that if the subset s_j is in fact the entire set of values $s_j = \mathcal{S}_j$, as is often the case, the variable X_j is said *not* to appear in the rule (14.2).

14.2.1 Market Basket Analysis

General approaches to solving (14.2) are discussed in Section 14.2.5. These can be quite useful in many applications. However, they are not feasible for the very large ($p \approx 10^4$, $N \approx 10^8$) commercial data bases to which market basket analysis is often applied. Several further simplifications of (14.2) are required. First, only two types of subsets are considered; either s_j consists of a *single* value of X_j, $s_j = v_{0j}$, or it consists of the entire set of values that X_j can assume, $s_j = \mathcal{S}_j$. This simplifies the problem (14.2) to finding subsets of the integers $\mathcal{J} \subset \{1, \ldots, p\}$, and corresponding values v_{0j}, $j \in \mathcal{J}$, such that

$$\Pr\left[\bigcap_{j\in\mathcal{J}}(X_j = v_{0j})\right] \tag{14.3}$$

is large. Figure 14.1 illustrates this assumption.

One can apply the technique of *dummy variables* to turn (14.3) into a problem involving only binary-valued variables. Here we assume that the support $\mathcal{S}_j$ is finite for each variable X_j. Specifically, a new set of variables $Z_1, \ldots, Z_K$ is created, one such variable for each of the values v_{lj} attainable by each of the original variables $X_1, \ldots, X_p$. The number of dummy variables K is

$$K = \sum_{j=1}^{p} |\mathcal{S}_j|,$$

where $|\mathcal{S}_j|$ is the number of distinct values attainable by X_j. Each dummy variable is assigned the value $Z_k = 1$ if the variable with which it is associated takes on the corresponding value to which Z_k is assigned, and $Z_k = 0$ otherwise. This transforms (14.3) to finding a subset of the integers $\mathcal{K} \subset \{1, \ldots, K\}$ such that

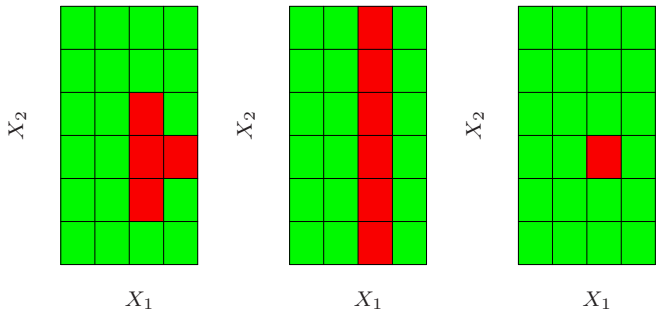

FIGURE 14.1. *Simplifications for association rules. Here there are two inputs X_1 and X_2, taking four and six distinct values, respectively. The red squares indicate areas of high density. To simplify the computations, we assume that the derived subset corresponds to either a single value of an input or all values. With this assumption we could find either the middle or right pattern, but not the left one.*

$$\Pr\left[\bigcap_{k\in\mathcal{K}}(Z_k = 1)\right] = \Pr\left[\prod_{k\in\mathcal{K}} Z_k = 1\right] \tag{14.4}$$

is large. This is the standard formulation of the market basket problem. The set $\mathcal{K}$ is called an "item set." The number of variables Z_k in the item set is called its "size" (note that the size is no bigger than p). The estimated value of (14.4) is taken to be the fraction of observations in the data base for which the conjunction in (14.4) is true:

$$\widehat{\Pr}\left[\prod_{k\in\mathcal{K}}(Z_k = 1)\right] = \frac{1}{N}\sum_{i=1}^{N}\prod_{k\in\mathcal{K}} z_{ik}. \tag{14.5}$$

Here z_{ik} is the value of Z_k for this ith case. This is called the "support" or "prevalence" $T(\mathcal{K})$ of the item set $\mathcal{K}$. An observation i for which $\prod_{k\in\mathcal{K}} z_{ik} = 1$ is said to "contain" the item set $\mathcal{K}$.

In association rule mining a lower support bound t is specified, and one seeks *all* item sets $\mathcal{K}_l$ that can be formed from the variables $Z_1, \ldots, Z_K$ with support in the data base greater than this lower bound t

$$\{\mathcal{K}_l | T(\mathcal{K}_l) > t\}. \tag{14.6}$$

14.2.2 The Apriori Algorithm

The solution to this problem (14.6) can be obtained with feasible computation for very large data bases provided the threshold t is adjusted so that (14.6) consists of only a small fraction of all 2^K possible item sets. The "Apriori" algorithm (Agrawal et al., 1995) exploits several aspects of the

curse of dimensionality to solve (14.6) with a small number of passes over the data. Specifically, for a given support threshold t:

- The cardinality $|\{\mathcal{K}|\, T(\mathcal{K}) > t\}|$ is relatively small.
- Any item set $\mathcal{L}$ consisting of a subset of the items in $\mathcal{K}$ must have support greater than or equal to that of $\mathcal{K}$, $\mathcal{L} \subseteq \mathcal{K} \Rightarrow T(\mathcal{L}) \geq T(\mathcal{K})$.

The first pass over the data computes the support of all single-item sets. Those whose support is less than the threshold are discarded. The second pass computes the support of all item sets of size two that can be formed from pairs of the single items surviving the first pass. In other words, to generate all frequent itemsets with $|\mathcal{K}| = m$, we need to consider only candidates such that *all* of their m ancestral item sets of size $m-1$ are frequent. Those size-two item sets with support less than the threshold are discarded. Each successive pass over the data considers only those item sets that can be formed by combining those that survived the previous pass with those retained from the first pass. Passes over the data continue until all candidate rules from the previous pass have support less than the specified threshold. The Apriori algorithm requires only one pass over the data for each value of $|\mathcal{K}|$, which is crucial since we assume the data cannot be fitted into a computer's main memory. If the data are sufficiently sparse (or if the threshold t is high enough), then the process will terminate in reasonable time even for huge data sets.

There are many additional tricks that can be used as part of this strategy to increase speed and convergence (Agrawal et al., 1995). The Apriori algorithm represents one of the major advances in data mining technology.

Each high support item set $\mathcal{K}$ (14.6) returned by the Apriori algorithm is cast into a set of "association rules." The items Z_k, $k \in \mathcal{K}$, are partitioned into two disjoint subsets, $A \cup B = \mathcal{K}$, and written

$$A \Rightarrow B. \tag{14.7}$$

The first item subset A is called the "antecedent" and the second B the "consequent." Association rules are defined to have several properties based on the prevalence of the antecedent and consequent item sets in the data base. The "support" of the rule $T(A \Rightarrow B)$ is the fraction of observations in the union of the antecedent and consequent, which is just the support of the item set $\mathcal{K}$ from which they were derived. It can be viewed as an estimate (14.5) of the probability of simultaneously observing both item sets $\Pr(A \text{ and } B)$ in a randomly selected market basket. The "confidence" or "predictability" $C(A \Rightarrow B)$ of the rule is its support divided by the support of the antecedent

$$C(A \Rightarrow B) = \frac{T(A \Rightarrow B)}{T(A)}, \tag{14.8}$$

which can be viewed as an estimate of $\Pr(B \,|\, A)$. The notation $\Pr(A)$, the probability of an item set A occurring in a basket, is an abbreviation for

$\Pr(\prod_{k\in A} Z_k = 1)$. The "expected confidence" is defined as the support of the consequent $T(B)$, which is an estimate of the unconditional probability $\Pr(B)$. Finally, the "lift" of the rule is defined as the confidence divided by the expected confidence

$$L(A \Rightarrow B) = \frac{C(A \Rightarrow B)}{T(B)}.$$

This is an estimate of the association measure $\Pr(A \text{ and } B)/\Pr(A)\Pr(B)$.

As an example, suppose the item set $\mathcal{K} = \{\texttt{peanut butter, jelly, bread}\}$ and consider the rule $\{\texttt{peanut butter, jelly}\} \Rightarrow \{\texttt{bread}\}$. A support value of 0.03 for this rule means that `peanut butter`, `jelly`, and `bread` appeared together in 3% of the market baskets. A confidence of 0.82 for this rule implies that when `peanut butter` and `jelly` were purchased, 82% of the time `bread` was also purchased. If bread appeared in 43% of all market baskets then the rule $\{\texttt{peanut butter, jelly}\} \Rightarrow \{\texttt{bread}\}$ would have a lift of 1.95.

The goal of this analysis is to produce association rules (14.7) with both high values of support and confidence (14.8). The Apriori algorithm returns all item sets with high support as defined by the support threshold t (14.6). A confidence threshold c is set, and all rules that can be formed from those item sets (14.6) with confidence greater than this value

$$\{A \Rightarrow B \,|\, C(A \Rightarrow B) > c\} \tag{14.9}$$

are reported. For each item set $\mathcal{K}$ of size $|\mathcal{K}|$ there are $2^{|\mathcal{K}|-1} - 1$ rules of the form $A \Rightarrow (\mathcal{K} - A)$, $A \subset \mathcal{K}$. Agrawal et al. (1995) present a variant of the Apriori algorithm that can rapidly determine which rules survive the confidence threshold (14.9) from all possible rules that can be formed from the solution item sets (14.6).

The output of the entire analysis is a collection of association rules (14.7) that satisfy the constraints

$$T(A \Rightarrow B) > t \quad \text{and} \quad C(A \Rightarrow B) > c.$$

These are generally stored in a data base that can be queried by the user. Typical requests might be to display the rules in sorted order of confidence, lift or support. More specifically, one might request such a list conditioned on particular items in the antecedent or especially the consequent. For example, a request might be the following:

> *Display all transactions in which ice skates are the consequent that have confidence over* 80% *and support of more than* 2%.

This could provide information on those items (antecedent) that predicate sales of ice skates. Focusing on a particular consequent casts the problem into the framework of supervised learning.

Association rules have become a popular tool for analyzing very large commercial data bases in settings where market basket is relevant. That is

when the data can be cast in the form of a multidimensional contingency table. The output is in the form of conjunctive rules (14.4) that are easily understood and interpreted. The Apriori algorithm allows this analysis to be applied to huge data bases, much larger that are amenable to other types of analyses. Association rules are among data mining's biggest successes.

Besides the restrictive form of the data to which they can be applied, association rules have other limitations. Critical to computational feasibility is the support threshold (14.6). The number of solution item sets, their size, and the number of passes required over the data can grow exponentially with decreasing size of this lower bound. Thus, rules with high confidence or lift, but low support, will not be discovered. For example, a high confidence rule such as `vodka` $\Rightarrow$ `caviar` will not be uncovered owing to the low sales volume of the consequent `caviar`.

14.2.3 Example: Market Basket Analysis

We illustrate the use of Apriori on a moderately sized demographics data base. This data set consists of $N = 9409$ questionnaires filled out by shopping mall customers in the San Francisco Bay Area (Impact Resources, Inc., Columbus OH, 1987). Here we use answers to the first 14 questions, relating to demographics, for illustration. These questions are listed in Table 14.1. The data are seen to consist of a mixture of ordinal and (unordered) categorical variables, many of the latter having more than a few values. There are many missing values.

We used a freeware implementation of the Apriori algorithm due to Christian Borgelt[1]. After removing observations with missing values, each ordinal predictor was cut at its median and coded by two dummy variables; each categorical predictor with k categories was coded by k dummy variables. This resulted in a 6876×50 matrix of 6876 observations on 50 dummy variables.

The algorithm found a total of 6288 association rules, involving ≤ 5 predictors, with support of at least 10%. Understanding this large set of rules is itself a challenging data analysis task. We will not attempt this here, but only illustrate in Figure 14.2 the relative frequency of each dummy variable in the data (top) and the association rules (bottom). Prevalent categories tend to appear more often in the rules, for example, the first category in language (English). However, others such as occupation are under-represented, with the exception of the first and fifth level.

Here are three examples of association rules found by the Apriori algorithm:

Association rule 1: Support 25%, confidence 99.7% and lift 1.03.

[1] See `http://fuzzy.cs.uni-magdeburg.de/~borgelt`.

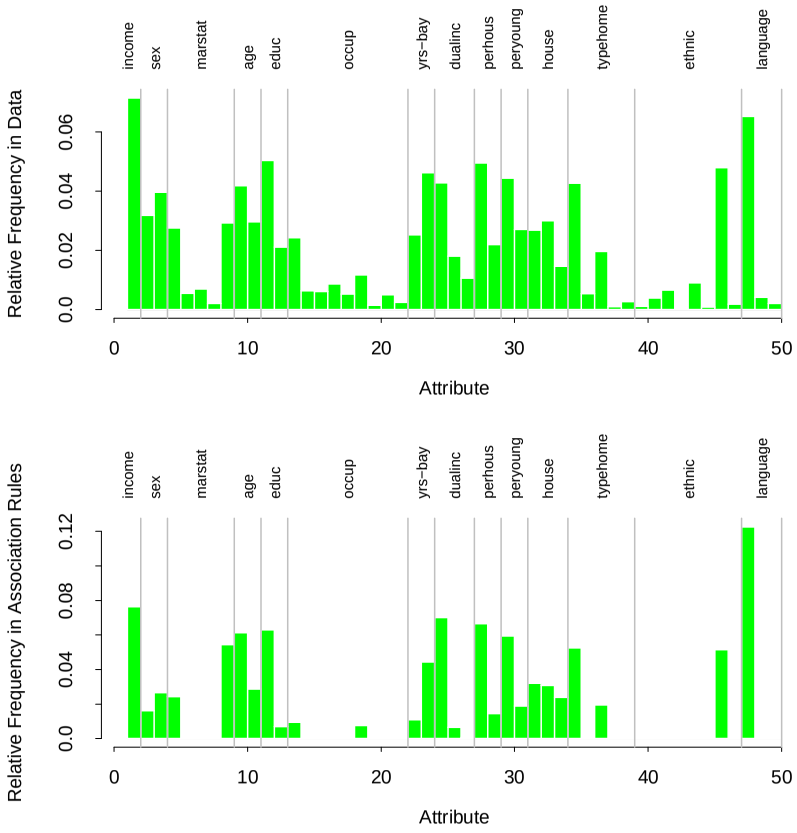

FIGURE 14.2. *Market basket analysis: relative frequency of each dummy variable (coding an input category) in the data (top), and the association rules found by the Apriori algorithm (bottom).*

TABLE 14.1. *Inputs for the demographic data.*

Feature	Demographic	# Values	Type
1	Sex	2	Categorical
2	Marital status	5	Categorical
3	Age	7	Ordinal
4	Education	6	Ordinal
5	Occupation	9	Categorical
6	Income	9	Ordinal
7	Years in Bay Area	5	Ordinal
8	Dual incomes	3	Categorical
9	Number in household	9	Ordinal
10	Number of children	9	Ordinal
11	Householder status	3	Categorical
12	Type of home	5	Categorical
13	Ethnic classification	8	Categorical
14	Language in home	3	Categorical

$$\begin{bmatrix} \text{number in household} & = & 1 \\ \text{number of children} & = & 0 \end{bmatrix}$$
$$\Downarrow$$
$$\text{language in home} = \textit{English}$$

Association rule 2: Support 13.4%, confidence 80.8%, and lift 2.13.

$$\begin{bmatrix} \text{language in home} & = & \textit{English} \\ \text{householder status} & = & \textit{own} \\ \text{occupation} & = & \{\textit{professional/managerial}\} \end{bmatrix}$$
$$\Downarrow$$
$$\text{income} \geq \textit{\$40,000}$$

Association rule 3: Support 26.5%, confidence 82.8% and lift 2.15.

$$\begin{bmatrix} \text{language in home} & = & \textit{English} \\ \text{income} & < & \textit{\$40,000} \\ \text{marital status} & = & \textit{not married} \\ \text{number of children} & = & 0 \end{bmatrix}$$
$$\Downarrow$$
$$\text{education} \notin \{\textit{college graduate, graduate study}\}$$

We chose the first and third rules based on their high support. The second rule is an association rule with a high-income consequent, and could be used to try to target high-income individuals.

As stated above, we created dummy variables for each category of the input predictors, for example, $Z_1 = I(\text{income} < \$40,000)$ and $Z_2 = I(\text{income} \geq \$40,000)$ for below and above the median income. If we were interested only in finding associations with the high-income category, we would include Z_2 but not Z_1. This is often the case in actual market basket problems, where we are interested in finding associations with the presence of a relatively rare item, but not associations with its absence.

14.2.4 Unsupervised as Supervised Learning

Here we discuss a technique for transforming the density estimation problem into one of supervised function approximation. This forms the basis for the generalized association rules described in the next section.

Let $g(x)$ be the unknown data probability density to be estimated, and $g_0(x)$ be a specified probability density function used for reference. For example, $g_0(x)$ might be the uniform density over the range of the variables. Other possibilities are discussed below. The data set $x_1, x_2, \ldots, x_N$ is presumed to be an *i.i.d.* random sample drawn from $g(x)$. A sample of size N_0 can be drawn from $g_0(x)$ using Monte Carlo methods. Pooling these two data sets, and assigning mass $w = N_0/(N + N_0)$ to those drawn from $g(x)$, and $w_0 = N/(N + N_0)$ to those drawn from $g_0(x)$, results in a random sample drawn from the mixture density $(g(x) + g_0(x))/2$. If one assigns the value $Y = 1$ to each sample point drawn from $g(x)$ and $Y = 0$ those drawn from $g_0(x)$, then

$$\begin{aligned} \mu(x) = E(Y \mid x) &= \frac{g(x)}{g(x) + g_0(x)} \\ &= \frac{g(x)/g_0(x)}{1 + g(x)/g_0(x)} \end{aligned} \tag{14.10}$$

can be estimated by supervised learning using the combined sample

$$(y_1, x_1), (y_2, x_2), \ldots, (y_{N+N_0}, x_{N+N_0}) \tag{14.11}$$

as training data. The resulting estimate $\hat{\mu}(x)$ can be inverted to provide an estimate for $g(x)$

$$\hat{g}(x) = g_0(x)\frac{\hat{\mu}(x)}{1 - \hat{\mu}(x)}. \tag{14.12}$$

Generalized versions of logistic regression (Section 4.4) are especially well suited for this application since the log-odds,

$$f(x) = \log \frac{g(x)}{g_0(x)}, \tag{14.13}$$

are estimated directly. In this case one has

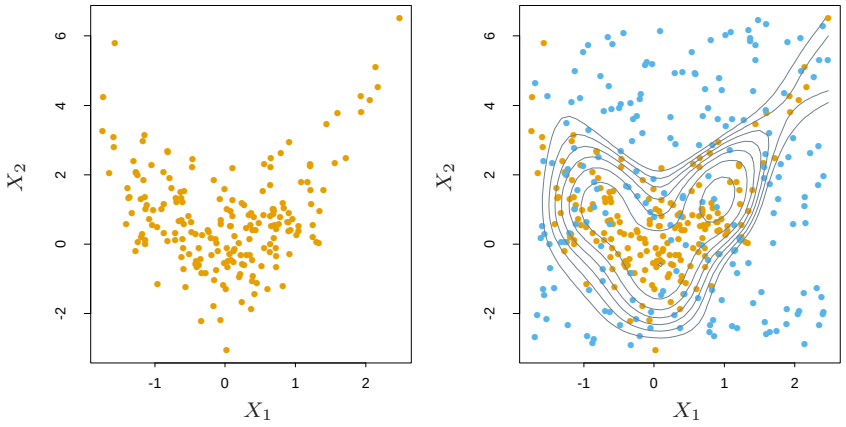

FIGURE 14.3. *Density estimation via classification. (Left panel:) Training set of* 200 *data points. (Right panel:) Training set plus* 200 *reference data points, generated uniformly over the rectangle containing the training data. The training sample was labeled as class* 1*, and the reference sample class* 0*, and a semiparametric logistic regression model was fit to the data. Some contours for* $\hat{g}(x)$ *are shown.*

$$\hat{g}(x) = g_0(x)\, e^{\hat{f}(x)}. \tag{14.14}$$

An example is shown in Figure 14.3. We generated a training set of size 200 shown in the left panel. The right panel shows the reference data (blue) generated uniformly over the rectangle containing the training data. The training sample was labeled as class 1, and the reference sample class 0, and a logistic regression model, using a tensor product of natural splines (Section 5.2.1), was fit to the data. Some probability contours of $\hat{\mu}(x)$ are shown in the right panel; these are also the contours of the density estimate $\hat{g}(x)$, since $\hat{g}(x) = \hat{\mu}(x)/(1-\hat{\mu}(x))$, is a monotone function. The contours roughly capture the data density.

In principle any reference density can be used for $g_0(x)$ in (14.14). In practice the accuracy of the estimate $\hat{g}(x)$ can depend greatly on particular choices. Good choices will depend on the data density $g(x)$ and the procedure used to estimate (14.10) or (14.13). If accuracy is the goal, $g_0(x)$ should be chosen so that the resulting functions $\mu(x)$ or $f(x)$ are approximated easily by the method being used. However, accuracy is not always the primary goal. Both $\mu(x)$ and $f(x)$ are monotonic functions of the density ratio $g(x)/g_0(x)$. They can thus be viewed as "contrast" statistics that provide information concerning departures of the data density $g(x)$ from the chosen reference density $g_0(x)$. Therefore, in data analytic settings, a choice for $g_0(x)$ is dictated by types of departures that are deemed most interesting in the context of the specific problem at hand. For example, if departures from uniformity are of interest, $g_0(x)$ might be the a uniform density over the range of the variables. If departures from joint normality

are of interest, a good choice for $g_0(x)$ would be a Gaussian distribution with the same mean vector and covariance matrix as the data. Departures from independence could be investigated by using

$$g_0(x) = \prod_{j=1}^{p} g_j(x_j), \tag{14.15}$$

where $g_j(x_j)$ is the marginal data density of X_j, the jth coordinate of X. A sample from this independent density (14.15) is easily generated from the data itself by applying a different random permutation to the data values of each of the variables.

As discussed above, unsupervised learning is concerned with revealing properties of the data density $g(x)$. Each technique focuses on a particular property or set of properties. Although this approach of transforming the problem to one of supervised learning (14.10)–(14.14) seems to have been part of the statistics folklore for some time, it does not appear to have had much impact despite its potential to bring well-developed supervised learning methodology to bear on unsupervised learning problems. One reason may be that the problem must be enlarged with a simulated data set generated by Monte Carlo techniques. Since the size of this data set should be at least as large as the data sample $N_0 \geq N$, the computation and memory requirements of the estimation procedure are at least doubled. Also, substantial computation may be required to generate the Monte Carlo sample itself. Although perhaps a deterrent in the past, these increased computational requirements are becoming much less of a burden as increased resources become routinely available. We illustrate the use of supervised learning methods for unsupervised learning in the next section.

14.2.5 Generalized Association Rules

The more general problem (14.2) of finding high-density regions in the data space can be addressed using the supervised learning approach described above. Although not applicable to the huge data bases for which market basket analysis is feasible, useful information can be obtained from moderately sized data sets. The problem (14.2) can be formulated as finding subsets of the integers $\mathcal{J} \subset \{1, 2, \ldots, p\}$ and corresponding value subsets s_j, $j \in \mathcal{J}$ for the corresponding variables X_j, such that

$$\widehat{\Pr}\left(\bigcap_{j\in\mathcal{J}} (X_j \in s_j)\right) = \frac{1}{N}\sum_{i=1}^{N} I\left(\bigcap_{j\in\mathcal{J}} (x_{ij} \in s_j)\right) \tag{14.16}$$

is large. Following the nomenclature of association rule analysis, $\{(X_j \in s_j)\}_{j\in\mathcal{J}}$ will be called a "generalized" item set. The subsets s_j corresponding to quantitative variables are taken to be contiguous intervals within

their range of values, and subsets for categorical variables can involve more than a single value. The ambitious nature of this formulation precludes a thorough search for all generalized item sets with support (14.16) greater than a specified minimum threshold, as was possible in the more restrictive setting of market basket analysis. Heuristic search methods must be employed, and the most one can hope for is to find a useful collection of such generalized item sets.

Both market basket analysis (14.5) and the generalized formulation (14.16) implicitly reference the uniform probability distribution. One seeks item sets that are more frequent than would be expected if all joint data values $(x_1, x_2, \ldots, x_N)$ were uniformly distributed. This favors the discovery of item sets whose marginal constituents $(X_j \in s_j)$ are *individually* frequent, that is, the quantity

$$\frac{1}{N}\sum_{i=1}^{N} I(x_{ij} \in s_j) \tag{14.17}$$

is large. Conjunctions of frequent subsets (14.17) will tend to appear more often among item sets of high support (14.16) than conjunctions of marginally less frequent subsets. This is why the rule `vodka` $\Rightarrow$ `caviar` is not likely to be discovered in spite of a high association (lift); neither item has high marginal support, so that their joint support is especially small. Reference to the uniform distribution can cause highly frequent item sets with low associations among their constituents to dominate the collection of highest support item sets.

Highly frequent subsets s_j are formed as disjunctions of the most frequent X_j-values. Using the product of the variable marginal data densities (14.15) as a reference distribution removes the preference for highly frequent values of the individual variables in the discovered item sets. This is because the density ratio $g(x)/g_0(x)$ is uniform if there are no associations among the variables (complete independence), regardless of the frequency distribution of the individual variable values. Rules like `vodka` $\Rightarrow$ `caviar` would have a chance to emerge. It is not clear however, how to incorporate reference distributions other than the uniform into the Apriori algorithm. As explained in Section 14.2.4, it is straightforward to generate a sample from the product density (14.15), given the original data set.

After choosing a reference distribution, and drawing a sample from it as in (14.11), one has a supervised learning problem with a binary-valued output variable $Y \in \{0, 1\}$. The goal is to use this training data to find regions

$$R = \bigcap_{j \in \mathcal{J}} (X_j \in s_j) \tag{14.18}$$

for which the target function $\mu(x) = E(Y \mid x)$ is relatively large. In addition, one might wish to require that the *data* support of these regions

$$T(R) = \int_{x \in R} g(x)\, dx \tag{14.19}$$

not be too small.

14.2.6 Choice of Supervised Learning Method

The regions (14.18) are defined by conjunctive rules. Hence supervised methods that learn such rules would be most appropriate in this context. The terminal nodes of a CART decision tree are defined by rules precisely of the form (14.18). Applying CART to the pooled data (14.11) will produce a decision tree that attempts to model the target (14.10) over the entire data space by a disjoint set of regions (terminal nodes). Each region is defined by a rule of the form (14.18). Those terminal nodes t with high average y-values

$$\bar{y}_t = \text{ave}(y_i \,|\, x_i \in t)$$

are candidates for high-support generalized item sets (14.16). The actual (data) support is given by

$$T(R) = \bar{y}_t \cdot \frac{N_t}{N + N_0},$$

where N_t is the number of (pooled) observations within the region represented by the terminal node. By examining the resulting decision tree, one might discover interesting generalized item sets of relatively high-support. These can then be partitioned into antecedents and consequents in a search for generalized association rules of high confidence and/or lift.

Another natural learning method for this purpose is the patient rule induction method PRIM described in Section 9.3. PRIM also produces rules precisely of the form (14.18), but it is especially designed for finding high-support regions that maximize the average target (14.10) value within them, rather than trying to model the target function over the entire data space. It also provides more control over the support/average-target-value tradeoff.

Exercise 14.3 addresses an issue that arises with either of these methods when we generate random data from the product of the marginal distributions.

14.2.7 Example: Market Basket Analysis (Continued)

We illustrate the use of PRIM on the demographics data of Table 14.1.

Three of the high-support generalized item sets emerging from the PRIM analysis were the following:

Item set 1: Support= 24%.

$$\left[\begin{array}{rcl} \text{marital status} & = & \textit{married} \\ \text{householder status} & = & \textit{own} \\ \text{type of home} & \neq & \textit{apartment} \end{array}\right]$$

Item set 2: Support= 24%.

$$\left[\begin{array}{rcl} \text{age} & \leq & 24 \\ \text{marital status} & \in & \{\textit{living together-not married, single}\} \\ \text{occupation} & \notin & \{\textit{professional, homemaker, retired}\} \\ \text{householder status} & \in & \{\textit{rent, live with family}\} \end{array}\right]$$

Item set 3: Support= 15%.

$$\left[\begin{array}{rcl} \text{householder status} & = & \textit{rent} \\ \text{type of home} & \neq & \textit{house} \\ \text{number in household} & \leq & 2 \\ \text{number of children} & = & 0 \\ \text{occupation} & \notin & \{\textit{homemaker, student, unemployed}\} \\ \text{income} & \in & [\$20{,}000, \$150{,}000] \end{array}\right]$$

Generalized association rules derived from these item sets with confidence (14.8) greater than 95% are the following:

Association rule 1: Support 25%, confidence 99.7% and lift 1.35.

$$\left[\begin{array}{rcl} \text{marital status} & = & \textit{married} \\ \text{householder status} & = & \textit{own} \end{array}\right] \Downarrow \text{type of home} \neq \textit{apartment}$$

Association rule 2: Support 25%, confidence 98.7% and lift 1.97.

$$\left[\begin{array}{rcl} \text{age} & \leq & 24 \\ \text{occupation} & \notin & \{\textit{professional, homemaker, retired}\} \\ \text{householder status} & \in & \{\textit{rent, live with family}\} \end{array}\right] \Downarrow \text{marital status} \in \{\textit{single, living together-not married}\}$$

Association rule 3: Support 25%, confidence 95.9% and lift 2.61.

$$\left[\begin{array}{rcl} \text{householder status} & = & \textit{own} \\ \text{type of home} & \neq & \textit{apartment} \end{array}\right] \Downarrow \text{marital status} = \textit{married}$$

Association rule 4: Support 15%, confidence 95.4% and lift 1.50.

$$\left[\begin{array}{rcl} \text{householder status} & = & \textit{rent} \\ \text{type of home} & \neq & \textit{house} \\ \text{number in household} & \leq & 2 \\ \text{occupation} & \notin & \{\textit{homemaker, student, unemployed}\} \\ \text{income} & \in & [\$20{,}000, \$150{,}000] \end{array}\right] \Downarrow \text{number of children} = 0$$

There are no great surprises among these particular rules. For the most part they verify intuition. In other contexts where there is less prior information available, unexpected results have a greater chance to emerge. These results do illustrate the type of information generalized association rules can provide, and that the supervised learning approach, coupled with a ruled induction method such as CART or PRIM, can uncover item sets exhibiting high associations among their constituents.

How do these generalized association rules compare to those found earlier by the Apriori algorithm? Since the Apriori procedure gives thousands of rules, it is difficult to compare them. However some general points can be made. The Apriori algorithm is exhaustive—it finds *all* rules with support greater than a specified amount. In contrast, PRIM is a greedy algorithm and is not guaranteed to give an "optimal" set of rules. On the other hand, the Apriori algorithm can deal only with dummy variables and hence could not find some of the above rules. For example, since `type of home` is a categorical input, with a dummy variable for each level, Apriori could not find a rule involving the set

$$\text{type of home} \neq \textit{apartment}.$$

To find this set, we would have to code a dummy variable for *apartment* versus the other categories of type of home. It will not generally be feasible to precode all such potentially interesting comparisons.

14.3 Cluster Analysis

Cluster analysis, also called data segmentation, has a variety of goals. All relate to grouping or segmenting a collection of objects into subsets or "clusters," such that those within each cluster are more closely related to one another than objects assigned to different clusters. An object can be described by a set of measurements, or by its relation to other objects. In addition, the goal is sometimes to arrange the clusters into a natural hierarchy. This involves successively grouping the clusters themselves so

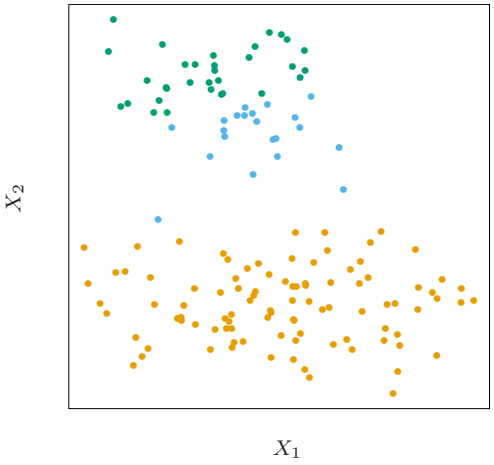

FIGURE 14.4. *Simulated data in the plane, clustered into three classes (represented by orange, blue and green) by the K-means clustering algorithm*

that at each level of the hierarchy, clusters within the same group are more similar to each other than those in different groups.

Cluster analysis is also used to form descriptive statistics to ascertain whether or not the data consists of a set distinct subgroups, each group representing objects with substantially different properties. This latter goal requires an assessment of the degree of difference between the objects assigned to the respective clusters.

Central to all of the goals of cluster analysis is the notion of the degree of similarity (or dissimilarity) between the individual objects being clustered. A clustering method attempts to group the objects based on the definition of similarity supplied to it. This can only come from subject matter considerations. The situation is somewhat similar to the specification of a loss or cost function in prediction problems (supervised learning). There the cost associated with an inaccurate prediction depends on considerations outside the data.

Figure 14.4 shows some simulated data clustered into three groups via the popular K-means algorithm. In this case two of the clusters are not well separated, so that "segmentation" more accurately describes the part of this process than "clustering." K-means clustering starts with guesses for the three cluster centers. Then it alternates the following steps until convergence:

- for each data point, the closest cluster center (in Euclidean distance) is identified;

- each cluster center is replaced by the coordinate-wise average of all data points that are closest to it.

We describe K-means clustering in more detail later, including the problem of how to choose the number of clusters (three in this example). K-means clustering is a *top-down* procedure, while other cluster approaches that we discuss are *bottom-up*. Fundamental to all clustering techniques is the choice of distance or dissimilarity measure between two objects. We first discuss distance measures before describing a variety of algorithms for clustering.

14.3.1 Proximity Matrices

Sometimes the data is represented directly in terms of the proximity (alikeness or affinity) between pairs of objects. These can be either *similarities* or *dissimilarities* (difference or lack of affinity). For example, in social science experiments, participants are asked to judge by how much certain objects differ from one another. Dissimilarities can then be computed by averaging over the collection of such judgments. This type of data can be represented by an $N \times N$ matrix $\mathbf{D}$, where N is the number of objects, and each element $d_{ii'}$ records the proximity between the ith and i'th objects. This matrix is then provided as input to the clustering algorithm.

Most algorithms presume a matrix of dissimilarities with nonnegative entries and zero diagonal elements: $d_{ii} = 0,\ i = 1, 2, \ldots, N$. If the original data were collected as similarities, a suitable monotone-decreasing function can be used to convert them to dissimilarities. Also, most algorithms assume symmetric dissimilarity matrices, so if the original matrix $\mathbf{D}$ is not symmetric it must be replaced by $(\mathbf{D}+\mathbf{D}^T)/2$. Subjectively judged dissimilarities are seldom *distances* in the strict sense, since the triangle inequality $d_{ii'} \le d_{ik}+d_{i'k}$, for all $k \in \{1, \ldots, N\}$ does not hold. Thus, some algorithms that assume distances cannot be used with such data.

14.3.2 Dissimilarities Based on Attributes

Most often we have measurements x_{ij} for $i = 1, 2, \ldots, N$, on variables $j = 1, 2, \ldots, p$ (also called *attributes*). Since most of the popular clustering algorithms take a dissimilarity matrix as their input, we must first construct pairwise dissimilarities between the observations. In the most common case, we define a dissimilarity $d_j(x_{ij}, x_{i'j})$ between values of the jth attribute, and then define

$$D(x_i, x_{i'}) = \sum_{j=1}^{p} d_j(x_{ij}, x_{i'j}) \tag{14.20}$$

as the dissimilarity between objects i and i'. By far the most common choice is squared distance

$$d_j(x_{ij}, x_{i'j}) = (x_{ij} - x_{i'j})^2. \tag{14.21}$$

However, other choices are possible, and can lead to potentially different results. For nonquantitative attributes (e.g., categorical data), squared distance may not be appropriate. In addition, it is sometimes desirable to weigh attributes differently rather than giving them equal weight as in (14.20).

We first discuss alternatives in terms of the attribute type:

Quantitative variables. Measurements of this type of variable or attribute are represented by continuous real-valued numbers. It is natural to define the "error" between them as a monotone-increasing function of their absolute difference

$$d(x_i, x_{i'}) = l(|x_i - x_{i'}|).$$

Besides squared-error loss $(x_i - x_{i'})^2$, a common choice is the identity (absolute error). The former places more emphasis on larger differences than smaller ones. Alternatively, clustering can be based on the correlation

$$\rho(x_i, x_{i'}) = \frac{\sum_j (x_{ij} - \bar{x}_i)(x_{i'j} - \bar{x}_{i'})}{\sqrt{\sum_j (x_{ij} - \bar{x}_i)^2 \sum_j (x_{i'j} - \bar{x}_{i'})^2}}, \tag{14.22}$$

with $\bar{x}_i = \sum_j x_{ij}/p$. Note that this is averaged over *variables*, not observations. If the *observations* are first standardized, then $\sum_j (x_{ij} - x_{i'j})^2 \propto 2(1 - \rho(x_i, x_{i'}))$. Hence clustering based on correlation (similarity) is equivalent to that based on squared distance (dissimilarity).

Ordinal variables. The values of this type of variable are often represented as contiguous integers, and the realizable values are considered to be an ordered set. Examples are academic grades (A, B, C, D, F), degree of preference (can't stand, dislike, OK, like, terrific). Rank data are a special kind of ordinal data. Error measures for ordinal variables are generally defined by replacing their M original values with

$$\frac{i - 1/2}{M}, \; i = 1, \ldots, M \tag{14.23}$$

in the prescribed order of their original values. They are then treated as quantitative variables on this scale.

Categorical variables. With unordered categorical (also called nominal) variables, the degree-of-difference between pairs of values must be delineated explicitly. If the variable assumes M distinct values, these can be arranged in a symmetric $M \times M$ matrix with elements $L_{rr'} = L_{r'r}, L_{rr} = 0, L_{rr'} \geq 0$. The most common choice is $L_{rr'} = 1$ for all $r \neq r'$, while unequal losses can be used to emphasize some errors more than others.

14.3.3 Object Dissimilarity

Next we define a procedure for combining the p-individual attribute dissimilarities $d_j(x_{ij}, x_{i'j})$, $j = 1, 2, \ldots, p$ into a single overall measure of dissimilarity $D(x_i, x_{i'})$ between two objects or observations $(x_i, x_{i'})$ possessing the respective attribute values. This is nearly always done by means of a weighted average (convex combination)

$$D(x_i, x_{i'}) = \sum_{j=1}^{p} w_j \cdot d_j(x_{ij}, x_{i'j}); \quad \sum_{j=1}^{p} w_j = 1. \tag{14.24}$$

Here w_j is a weight assigned to the jth attribute regulating the relative influence of that variable in determining the overall dissimilarity between objects. This choice should be based on subject matter considerations.

It is important to realize that setting the weight w_j to the same value for each variable (say, $w_j = 1 \; \forall \; j$) does *not* necessarily give all attributes equal influence. The influence of the jth attribute X_j on object dissimilarity $D(x_i, x_{i'})$ (14.24) depends upon its relative contribution to the average object dissimilarity measure over all pairs of observations in the data set

$$\bar{D} = \frac{1}{N^2} \sum_{i=1}^{N} \sum_{i'=1}^{N} D(x_i, x_{i'}) = \sum_{j=1}^{p} w_j \cdot \bar{d}_j,$$

with

$$\bar{d}_j = \frac{1}{N^2} \sum_{i=1}^{N} \sum_{i'=1}^{N} d_j(x_{ij}, x_{i'j}) \tag{14.25}$$

being the average dissimilarity on the jth attribute. Thus, the relative influence of the jth variable is $w_j \cdot \bar{d}_j$, and setting $w_j \sim 1/\bar{d}_j$ would give all attributes equal influence in characterizing overall dissimilarity between objects. For example, with p quantitative variables and squared-error distance used for each coordinate, then (14.24) becomes the (weighted) squared Euclidean distance

$$D_I(x_i, x_{i'}) = \sum_{j=1}^{p} w_j \cdot (x_{ij} - x_{i'j})^2 \tag{14.26}$$

between pairs of points in an $\mathbb{R}^p$, with the quantitative variables as axes. In this case (14.25) becomes

$$\bar{d}_j = \frac{1}{N^2} \sum_{i=1}^{N} \sum_{i'=1}^{N} (x_{ij} - x_{i'j})^2 = 2 \cdot \mathrm{var}_j, \tag{14.27}$$

where var_j is the sample estimate of $\mathrm{Var}(X_j)$. Thus, the relative importance of each such variable is proportional to its variance over the data

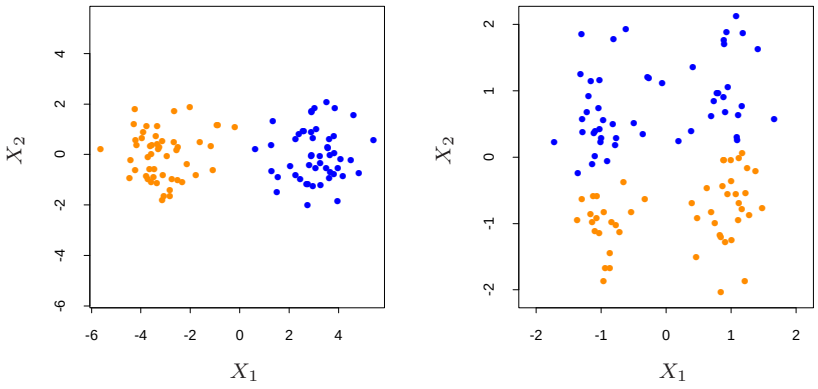

FIGURE 14.5. *Simulated data: on the left, K-means clustering (with K=2) has been applied to the raw data. The two colors indicate the cluster memberships. On the right, the features were first standardized before clustering. This is equivalent to using feature weights* $1/[2 \cdot \text{var}(X_j)]$. *The standardization has obscured the two well-separated groups. Note that each plot uses the same units in the horizontal and vertical axes.*

set. In general, setting $w_j = 1/\bar{d}_j$ for all attributes, irrespective of type, will cause each one of them to equally influence the overall dissimilarity between pairs of objects $(x_i, x_{i'})$. Although this may seem reasonable, and is often recommended, it can be highly counterproductive. If the goal is to segment the data into groups of similar objects, all attributes may not contribute equally to the (problem-dependent) notion of dissimilarity between objects. Some attribute value differences may reflect greater actual object dissimilarity in the context of the problem domain.

If the goal is to discover natural groupings in the data, some attributes may exhibit more of a grouping tendency than others. Variables that are more relevant in separating the groups should be assigned a higher influence in defining object dissimilarity. Giving all attributes equal influence in this case will tend to obscure the groups to the point where a clustering algorithm cannot uncover them. Figure 14.5 shows an example.

Although simple generic prescriptions for choosing the individual attribute dissimilarities $d_j(x_{ij}, x_{i'j})$ and their weights w_j can be comforting, there is no substitute for careful thought in the context of each individual problem. Specifying an appropriate dissimilarity measure is far more important in obtaining success with clustering than choice of clustering algorithm. This aspect of the problem is emphasized less in the clustering literature than the algorithms themselves, since it depends on domain knowledge specifics and is less amenable to general research.

Finally, often observations have *missing values* in one or more of the attributes. The most common method of incorporating missing values in dissimilarity calculations (14.24) is to omit each observation pair $x_{ij}, x_{i'j}$ having at least one value missing, when computing the dissimilarity between observations x_i and x'_i. This method can fail in the circumstance when both observations have no measured values in common. In this case both observations could be deleted from the analysis. Alternatively, the missing values could be imputed using the mean or median of each attribute over the nonmissing data. For categorical variables, one could consider the value "missing" as just another categorical value, if it were reasonable to consider two objects as being similar if they both have missing values on the same variables.

14.3.4 Clustering Algorithms

The goal of cluster analysis is to partition the observations into groups ("clusters") so that the pairwise dissimilarities between those assigned to the same cluster tend to be smaller than those in different clusters. Clustering algorithms fall into three distinct types: combinatorial algorithms, mixture modeling, and mode seeking.

Combinatorial algorithms work directly on the observed data with no direct reference to an underlying probability model. *Mixture modeling* supposes that the data is an *i.i.d* sample from some population described by a probability density function. This density function is characterized by a parameterized model taken to be a mixture of component density functions; each component density describes one of the clusters. This model is then fit to the data by maximum likelihood or corresponding Bayesian approaches. *Mode seekers* ("bump hunters") take a nonparametric perspective, attempting to directly estimate distinct modes of the probability density function. Observations "closest" to each respective mode then define the individual clusters.

Mixture modeling is described in Section 6.8. The PRIM algorithm, discussed in Sections 9.3 and 14.2.5, is an example of mode seeking or "bump hunting." We discuss combinatorial algorithms next.

14.3.5 Combinatorial Algorithms

The most popular clustering algorithms directly assign each observation to a group or cluster without regard to a probability model describing the data. Each observation is uniquely labeled by an integer $i \in \{1, \cdots, N\}$. A prespecified number of clusters $K < N$ is postulated, and each one is labeled by an integer $k \in \{1, \ldots, K\}$. Each observation is assigned to one and only one cluster. These assignments can be characterized by a many-to-one mapping, or *encoder* $k = C(i)$, that assigns the ith observation to the kth cluster. One seeks the particular encoder $C^*(i)$ that achieves the

required goal (details below), based on the dissimilarities $d(x_i, x_{i'})$ between every pair of observations. These are specified by the user as described above. Generally, the encoder $C(i)$ is explicitly delineated by giving its value (cluster assignment) for each observation i. Thus, the "parameters" of the procedure are the individual cluster assignments for each of the N observations. These are adjusted so as to *minimize* a "loss" function that characterizes the degree to which the clustering goal is *not* met.

One approach is to directly specify a mathematical loss function and attempt to minimize it through some combinatorial optimization algorithm. Since the goal is to assign close points to the same cluster, a natural loss (or "energy") function would be

$$W(C) = \frac{1}{2}\sum_{k=1}^{K}\sum_{C(i)=k}\sum_{C(i')=k} d(x_i, x_{i'}). \tag{14.28}$$

This criterion characterizes the extent to which observations assigned to the same cluster tend to be close to one another. It is sometimes referred to as the "within cluster" point scatter since

$$T = \frac{1}{2}\sum_{i=1}^{N}\sum_{i'=1}^{N} d_{ii'} = \frac{1}{2}\sum_{k=1}^{K}\sum_{C(i)=k}\left(\sum_{C(i')=k} d_{ii'} + \sum_{C(i')\neq k} d_{ii'}\right),$$

or

$$T = W(C) + B(C),$$

where $d_{ii'} = d(x_i, x_{i'})$. Here T is the *total* point scatter, which is a constant given the data, independent of cluster assignment. The quantity

$$B(C) = \frac{1}{2}\sum_{k=1}^{K}\sum_{C(i)=k}\sum_{C(i')\neq k} d_{ii'} \tag{14.29}$$

is the *between-cluster* point scatter. This will tend to be large when observations assigned to different clusters are far apart. Thus one has

$$W(C) = T - B(C)$$

and minimizing $W(C)$ is equivalent to *maximizing* $B(C)$.

Cluster analysis by combinatorial optimization is straightforward in principle. One simply minimizes W or equivalently maximizes B over all possible assignments of the N data points to K clusters. Unfortunately, such optimization by complete enumeration is feasible only for very small data sets. The number of distinct assignments is (Jain and Dubes, 1988)

$$S(N, K) = \frac{1}{K!}\sum_{k=1}^{K}(-1)^{K-k}\binom{K}{k}k^N. \tag{14.30}$$

For example, $S(10, 4) = 34,105$ which is quite feasible. But, $S(N, K)$ grows very rapidly with increasing values of its arguments. Already $S(19, 4) \simeq$

10^{10}, and most clustering problems involve much larger data sets than $N = 19$. For this reason, practical clustering algorithms are able to examine only a very small fraction of all possible encoders $k = C(i)$. The goal is to identify a small subset that is likely to contain the optimal one, or at least a good suboptimal partition.

Such feasible strategies are based on iterative greedy descent. An initial partition is specified. At each iterative step, the cluster assignments are changed in such a way that the value of the criterion is improved from its previous value. Clustering algorithms of this type differ in their prescriptions for modifying the cluster assignments at each iteration. When the prescription is unable to provide an improvement, the algorithm terminates with the current assignments as its solution. Since the assignment of observations to clusters at any iteration is a perturbation of that for the previous iteration, only a very small fraction of all possible assignments (14.30) are examined. However, these algorithms converge to *local* optima which may be highly suboptimal when compared to the global optimum.

14.3.6 K-means

The K-means algorithm is one of the most popular iterative descent clustering methods. It is intended for situations in which all variables are of the quantitative type, and squared Euclidean distance

$$d(x_i, x_{i'}) = \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2 = ||x_i - x_{i'}||^2$$

is chosen as the dissimilarity measure. Note that weighted Euclidean distance can be used by redefining the x_{ij} values (Exercise 14.1).

The within-point scatter (14.28) can be written as

$$\begin{aligned} W(C) &= \frac{1}{2} \sum_{k=1}^{K} \sum_{C(i)=k} \sum_{C(i')=k} ||x_i - x_{i'}||^2 \\ &= \sum_{k=1}^{K} N_k \sum_{C(i)=k} ||x_i - \bar{x}_k||^2, \end{aligned} \tag{14.31}$$

where $\bar{x}_k = (\bar{x}_{1k}, \ldots, \bar{x}_{pk})$ is the mean vector associated with the kth cluster, and $N_k = \sum_{i=1}^{N} I(C(i) = k)$. Thus, the criterion is minimized by assigning the N observations to the K clusters in such a way that within each cluster the average dissimilarity of the observations from the cluster mean, as defined by the points in that cluster, is minimized.

An iterative descent algorithm for solving

Algorithm 14.1 *K-means Clustering.*

1. For a given cluster assignment C, the total cluster variance (14.33) is minimized with respect to $\{m_1, \ldots, m_K\}$ yielding the means of the currently assigned clusters (14.32).

2. Given a current set of means $\{m_1, \ldots, m_K\}$, (14.33) is minimized by assigning each observation to the closest (current) cluster mean. That is,
$$C(i) = \underset{1 \le k \le K}{\operatorname{argmin}} ||x_i - m_k||^2. \tag{14.34}$$

3. Steps 1 and 2 are iterated until the assignments do not change.

$$C^* = \min_C \sum_{k=1}^{K} N_k \sum_{C(i)=k} ||x_i - \bar{x}_k||^2$$

can be obtained by noting that for any set of observations S

$$\bar{x}_S = \underset{m}{\operatorname{argmin}} \sum_{i \in S} ||x_i - m||^2. \tag{14.32}$$

Hence we can obtain C^* by solving the enlarged optimization problem

$$\min_{C, \{m_k\}_1^K} \sum_{k=1}^{K} N_k \sum_{C(i)=k} ||x_i - m_k||^2. \tag{14.33}$$

This can be minimized by an alternating optimization procedure given in Algorithm 14.1.

Each of steps 1 and 2 reduces the value of the criterion (14.33), so that convergence is assured. However, the result may represent a suboptimal local minimum. The algorithm of Hartigan and Wong (1979) goes further, and ensures that there is no single switch of an observation from one group to another group that will decrease the objective. In addition, one should start the algorithm with many different random choices for the starting means, and choose the solution having smallest value of the objective function.

Figure 14.6 shows some of the K-means iterations for the simulated data of Figure 14.4. The centroids are depicted by "O"s. The straight lines show the partitioning of points, each sector being the set of points closest to each centroid. This partitioning is called the *Voronoi tessellation*. After 20 iterations the procedure has converged.

14.3.7 Gaussian Mixtures as Soft K-means Clustering

The K-means clustering procedure is closely related to the EM algorithm for estimating a certain Gaussian mixture model. (Sections 6.8 and 8.5.1).

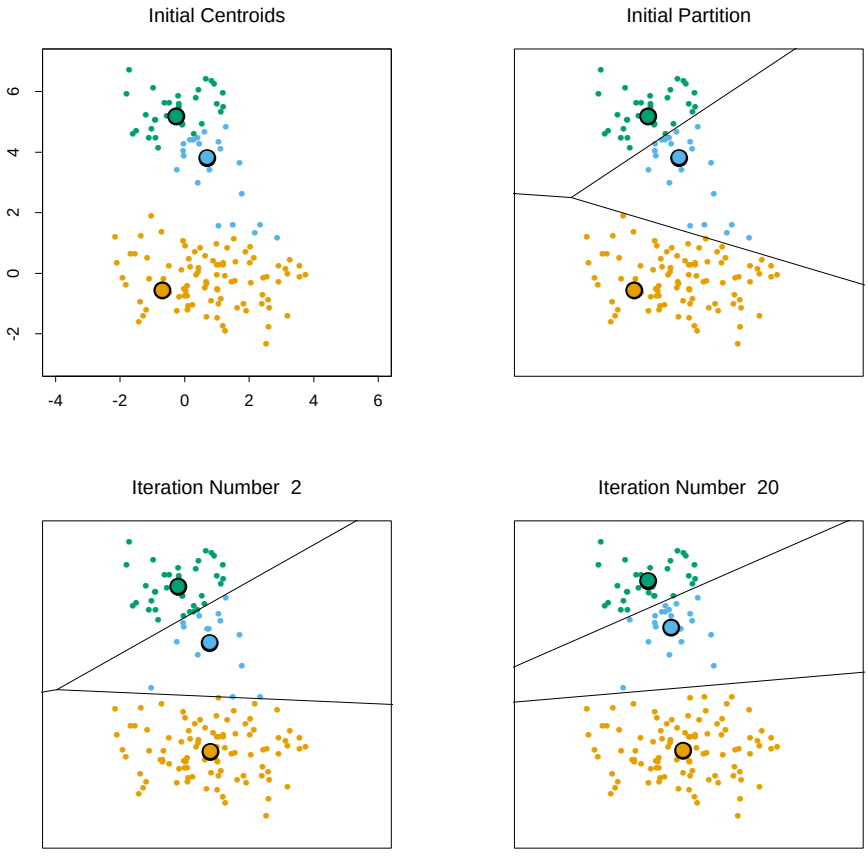

FIGURE 14.6. *Successive iterations of the K-means clustering algorithm for the simulated data of Figure 14.4.*